Mestrado em Engenharia Biomédica

2024/2025

Unidade Curricular: Sistemas Distribuídos e Segurança Informática

Docentes:

Jorge Pinto Leite

Pedro Rodrigues

Autores:

Ana Sofia Pinto[1], Camila Hayashida[2], Lícia Almeida[3], Mariana Pinto[4]

[1]1211040@isep.ipp.pt

[2]1211843@isep.ipp.pt

[3]1211459@isep.ipp.pt

[4]1210999@isep.ipp.pt

Índice

1. Introdução 3
1.1 Linux 3
1.2 Modelos OSI e TCP/IP e Protocolos 3
1.2.1 Modelos OSI e TCP/IP 3
1.2.1 Protocolo TCP 4
1.2.2 Protocolo HTTP 4
1.2.3 Protocolo FTP 5
1.2.4 Protocolo ARP 5
1.2.5 Protocolo ICMP 5
1.2.6 Protocolo POP3 6
1.2.7 Protocolo SMB2 6
1.2.8 Protocolo DCE/RPC 7
1.2.9 Protocolo SVCCTL 7
1.2.10. Protocolo UDP 8
1.3 Análise de Tráfego de Rede 8
1.4 Wireshark 8
2. Capturas 9
2.1 Captura 1 10
2.1.1 Protocolo Principal 10
2.1.2 Análise das Máquinas envolvidas na Captura 10
2.2 Captura 2 12
2.2.1 Protocolo Principal 12
2.2.2 Análise das Máquinas envolvidas na Captura 12
2.2.3 Comandos e Funcionalidades solicitadas 13
2.3 Captura 3 17
2.3.1 Protocolo Principal 17
2.3.2 Análise das Máquinas envolvidas na Captura 18
2.3.3 Comandos e Funcionalidades solicitadas 19
2.4 Captura 4 33
2.4.1 Protocolo Principal 33
2.4.2 Análise das Máquinas envolvidas na Captura 34
2.4.3 Comandos e Funcionalidades solicitadas 34
2.5 Captura 5 39
2.6 Captura 6 43
3. Conclusão 47

1. Introdução

1.1 Linux

Linus Torvalds desenvolveu a primeira versão do Linux, em 1991, como um sistema operativo *open source* para computadores [1]. Um sistema operativo trata-se de um *software* que gere diretamente os recursos e o *hardware* de um sistema, como CPU, memória e armazenamento. Este intermedeia a comunicação entre as aplicações e o *hardware*, conectando todos os seus programas aos recursos físicos que executam tarefas. O Linux foi inicialmente concebido para ser semelhante ao UNIX, mas ao longo do tempo evoluiu de forma significativa e, atualmente, é executado numa vasta gama de *hardwares*, desde *smartphones* até supercomputadores. Qualquer sistema operativo baseado em Linux inclui o *kernel* Linux, responsável pela gestão dos recursos de hardware, e um conjunto de pacotes de *software*, que constitui o restante do sistema [2]. Atualmente, o Linux é um sistema operativo completo, compatível com diversas arquiteturas, incluindo *Alpha*, ARM, *PowerPC*, SPARC, x86-64, entre outras. O interesse comercial pelo Linux é robusto, com novas empresas especializadas, como a *Red Hat*, bem como grandes corporações estabelecidas, como a IBM, a oferecerem soluções baseadas em Linux para responder às exigências modernas de empresas e centros de dados [1].

1.2 Modelos OSI e TCP/IP e Protocolos

Nesta secção, são descritos os protocolos presentes nas diferentes capturas, com base nos modelos de referência OSI e TCP/IP. Modelos estes que fornecem uma estrutura para entender como a comunicação entre sistemas ocorre numa rede, dividindo o processo em camadas específicas, cada uma com as suas funções e responsabilidades.

1.2.1 Modelos OSI e TCP/IP

O modelo OSI (*Open Systems Interconnection*) foi desenvolvido pela ISO para padronizar e facilitar a comunicação entre diferentes sistemas de rede. Este modelo é composto por sete camadas hierárquicas, cada uma com funções específicas que vão desde a transmissão física de dados até a interação com o utilizador. As camadas, que incluem *Physical*, *Data Link*, *Network*, *Transport*, *Session*, *Presentation* e *Application* (Fig. 1), trabalham em conjunto para assegurar uma

transmissão de dados confiável e organizada, cada uma proporcionando serviços específicos e independentes para a camada superior. Já o modelo TCP/IP, base da internet moderna, é composto por quatro camadas: *Link*, *Internet*, *Transport* e *Application* (Fig. 1), destacando-se pela sua robustez e flexibilidade, tendo sido projetado para suportar falhas na infraestrutura e permitir a interconexão de múltiplas redes. A arquitetura TCP/IP oferece uma abordagem prática com foco na confiabilidade e na eficiência, permitindo que dados sejam transferidos de forma independente e não ordenada pela rede, com protocolos como IP para encaminhamento e TCP para controlo de fluxo e confiabilidade [3].

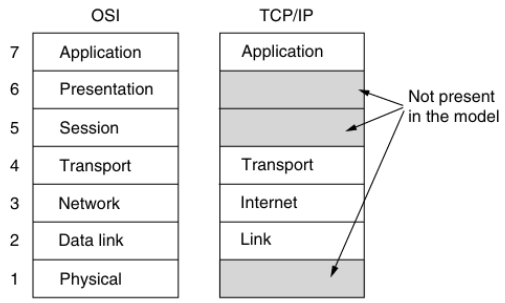


Figura 1 – Modelo OSI e Modelo TCP/IP *[3]*.

1.2.1 Protocolo TCP

O protocolo TCP (*Transmission Control Protocol*, ou em português – Protocolo de Controlo de Transmissão) é um dos principais protocolos do conjunto TCP/IP, posicionado na camada Transporte do modelo TCP/IP, semelhante à camada Transporte no modelo OSI, e oferece serviços de entrega de dados confiáveis, assegurando que as informações são transmitidas com precisão e na sequência correta, e impede que um remetente rápido sobrecarregue um recetor lento com mais dados do que ele pode processar [3].

1.2.2 Protocolo HTTP

O protocolo HTTP tem como principal tarefa transferir documentos web dos servidores para os clientes, e possivelmente também na direção oposta. O funcionamento da *World Wide Web* é baseado neste protocolo. Este protocolo define como é que as mensagens são formatadas e estabelecidas, para além de estabelecer regras sobre como os servidores devem responder a diferentes comandos. Os métodos HTTP mais conhecidos são *GET*, *POST*, *PUT* e *DELETE* [4].

1.2.3 Protocolo FTP

O FTP (*File Transfer Protocol*, ou em português - Protocolo de Transferência de Ficheiros) é utilizado para transferência de arquivos de um sistema para outro, controlada pelo utilizador, e, atua na camada de Aplicação do modelo OSI. Ao contrário de protocolos como HTTP, que lida com hipertextos e conteúdo *web*, o FTP foca-se exclusivamente em gerir e enviar arquivos de texto, binários e imagens. A transferência ocorre em duas fases: uma conexão TCP é estabelecida para trocar mensagens de controlo, como autenticação e especificação de arquivos, e uma segunda conexão TCP é usada para transferir os dados, sem existir sobrecarga de informações de controlo. Após a transferência, a conexão de controlo sinaliza a conclusão e aceita novos comandos [5].

1.2.4 Protocolo ARP

O protocolo ARP (*Adress Resolution Protocol*) é habitualmente utilizado em redes locais (LANs) para traduzir endereços IP em endereços MAC. Os endereços IP são atribuídos por administradores de rede em vários níveis de gerenciamento de rede, enquanto os endereços MAC são definidos por hardware e, portanto, são imutáveis e atribuídos pelos fabricantes de hardware. Para a comunicação, entretanto, ambos os tipos de endereços são necessários [4].

1.2.5 Protocolo ICMP

O ICMP é um protocolo que os dispositivos de uma rede utilizam para comunicar problemas com a transmissão de dados. Assim, uma das principais razões pela qual o ICMP é utilizado é para determinar se os dados estão a chegar ao seu destino e no momento certo. Para além disto, o ICMP pode também ser utilizado para realizar ataques de negação de serviço distribuído (DDoS). O modo de funcionamento do ICMP funciona da seguinte forma: ele transmite mensagens do recetor para o remetente sobre os dados que deveriam chegar. Caso os dados não cheguem ao destino ou sejam recebidos na ordem errada, o ICMP informa o remetente de modo que os dados possam ser reenviados. Desta forma, o ICMP é um protocolo que é utilizado especialmente para relatar erros, sendo, portanto, um protocolo para comunicar informações sobre dados, mas não para os gerir. Além disso, este protocolo não pertence a nenhum nível do modelo OSI, não estando também associado ao TCP ou UDP, logo, não existe a necessidade de um dispositivo se conectar a outro antes de enviar uma mensagem ICMP [6] .

1.2.6 Protocolo POP3

O POP (*Post Office Protocol*) refere-se a um protocolo que permite aceder remotamente a uma caixa de correio eletrónico (email). Utilizando este protocolo, as mensagens guardadas no servidor são transferidas para o armazenamento local do cliente de email, e removidas da caixa de correio pessoal no servidor [7]. Assim POP3 trata-se de um protocolo TCP / IP que permite que um cliente descarregue um email a partir de um servidor. Com este protocolo de leitura, é permitido fazer a transferência sequencial de mensagens do servidor para um cliente de email (Oulook, ThunderBird, etc) configurado noutro(s) dispositivo(s), por exemplo, computador, smartphone, tablet. Assim, o utilizador poderá ler, responder, apagar ou armazenar as mensagens na sua caixa de correio [8].

Durante uma sessão POP3, o servidor autentica o utilizador, verificando a identidade do mesmo, para determinar a que caixa de correio poderá aceder. Após a autenticação, o cliente pode consultar o número de mensagens na caixa de correio, visualizar os tamanhos das mensagens, recuperá-las e, se necessário, eliminá-las do servidor. Este protocolo opera num modelo cliente-servidor e utiliza o protocolo TCP na camada de transporte para assegurar a fiabilidade na transferência dos dados [4].

1.2.7 Protocolo SMB2

O *Server Message Block* (SMB) trata-se de um protocolo de comunicação cliente-servidor utilizado para partilhar o acesso a ficheiros, impressoras, portas seriais e outros recursos numa rede. O SMB transmite várias mensagens entre um cliente e um servidor para estabelecer uma ligação, permitindo que o cliente aceda a ficheiros e outros recursos, podendo realizar operações como abrir, ler e editar ficheiros num servidor remoto ou comunicar com qualquer programa de servidor configurado para receber pedidos SMB. O cliente envia pedidos por comandos e o servidor responde, o que concede permissões amplas ao sistema, tornando-o um alvo para atacantes, principalmente devido à sua capacidade de acesso remoto [9]. O protocolo SMB pode ser utilizado sobre seu protocolo TCP/IP ou outros protocolos de rede [10].

1.2.8 Protocolo DCE/RPC

O protocolo DCE/RPC (*Distributed Computing Environment / Remote Procedure Calls*) trata-se de uma especificação para um mecanismo de chamada de procedimento remoto que define APIs e um protocolo pela rede [11]. Este permite que uma aplicação num computador consiga executar procedimentos noutro computador, seguindo o paradigma cliente-servidor. No DCE/RPC, o programa que faz a chamada é o cliente e o programa que executa o procedimento remoto é o servidor. Assim, um cliente pode realizar uma chamada remota para um servidor, por exemplo, para inserir um pedido numa base de dados. Este é responsável pelo sucesso da execução no servidor e aguarda (bloqueia) até receber uma resposta. Neste caso, a resposta do servidor confirmaria a inserção do pedido na base de dados, replicando o comportamento de uma chamada de procedimento local. No entanto, a confiabilidade das chamadas remotas não é tão alta quanto em sistemas de processamento de transações em fila, devido à dependência da rede. O programador do cliente é responsável por lidar com possíveis falhas, como a indisponibilidade de rede ou do servidor, e falhas na execução do procedimento remoto. Isso torna a programação com DCE/RPC mais complexa do que as chamadas locais, pois exige tratamento adicional de erros para garantir o funcionamento correto [12].

1.2.9 Protocolo SVCCTL

O SVCCTL (*Service Control Manager Remote Protocol*) trata-se de um protocolo cliente/servidor utilizado para configurar e controlar programas de serviço que estão a ser executados num computador remoto. Uma sessão de gestão remota de serviços começa com o cliente a iniciar um pedido de conexão ao servidor. Se este aceitar o pedido, a conexão é estabelecida, permitindo que o cliente faça vários pedidos para modificar, consultar a configuração ou iniciar e parar serviços no servidor, utilizando a mesma sessão até que esta seja terminada [13].

Numa sessão típica deste protocolo, o cliente conecta-se ao servidor e solicita a abertura do *Service Control Manager* (SCM). Se o pedido for aceite, o servidor responde com um identificador de contexto RPC, que o cliente utiliza para operar no servidor. O cliente envia então novos pedidos especificando o tipo de operação (como iniciar, parar ou configurar serviços) e os parâmetros necessários. O servidor, ao aceitar o pedido, tenta realizar a operação e devolve o resultado ao cliente. Quando o cliente termina as operações, encerra a sessão enviando um pedido para fechar o identificador de contexto RPC [13].

1.2.10. Protocolo UDP

O protocolo UDP é um protocolo de troca de mensagens não fiável, ou seja, não estabelece uma conexão ao enviar dados. Este envia os dados sem confirmação de recebimento ou verificação de erros, podendo ocorrer a perda de alguns ou todos os dados durante a transmissão [14].

1.3 Análise de Tráfego de Rede

A análise de tráfego de rede é o processo de monitorização e inspeção dos padrões de tráfego numa rede. Ao identificar anomalias ou comportamentos suspeitos, esta pode auxiliar as equipas de segurança e operações de rede na deteção de ameaças. Além disso, esta análise é útil para resolver problemas de desempenho, como elevadas taxas de perda de pacotes ou latência de rede elevada. A sua importância reside no facto de os padrões de tráfego poderem variar significativamente. Os pacotes, ao deslocarem-se entre segmentos de rede e pontos de acesso, podem seguir rotas distintas, o que impacta o desempenho da rede conforme a eficiência dessas rotas. Além disso, atividades maliciosas, como varredura de portas ou ataques de negação de serviço (DoS), tendem a gerar padrões de tráfego anormais. Ao detetar estas anomalias, as organizações podem identificar potenciais riscos de segurança e bloqueá-los antes que ocorram violações. O funcionamento da análise de tráfego de rede envolve várias etapas. Primeiramente, a rede é monitorizada durante períodos de atividade normal para compreender os padrões típicos de tráfego de rede. De seguida, ocorre a identificação de anomalias e a sua posterior análise. Estas podem resultar de uma causa benigna, como a implantação de uma nova aplicação, resultando em mudanças no padrão de tráfego, ou de um problema, podendo estes ser relativos ao desempenho (como um router ou *switch* com falha, que leva os pacotes a serem encaminhados de forma inesperada) ou problemas de segurança, como uma varredura de rede por agentes maliciosos [15].

1.4 Wireshark

O *Wireshark* é um *software open-source*, gratuito, com uma interface gráfica de análise de pacotes, escrito em linguagem de programação C e distribuído sob a Licença Pública Geral (GPL). Criado pelo cientista Gerald Combs no final de 1997, foi concebido para diagnosticar e identificar

problemas de rede e monitorizar o tráfego de dados, capturando pacotes em redes ao vivo e permitindo a análise de ficheiros capturados previamente [16]. Além disso, fornece informações detalhadas sobre pacotes de rede, *frames Ethernet* e protocolos, incluindo um gráfico de I/O que resume o fluxo de pacotes e oferece insights especializados sobre a atividade da rede [17].

É considerada uma excelente ferramenta de *sniffing* numa rede de computadores, bem como a sua capacidade de detetar vários ataques em redes LAN como envenenamento ARP, ataques de DoS, inundação de MAC e *spoofing* de DNS, além de medidas de mitigação para esses ataques. Contudo, não é capaz de alertar os utilizadores sobre um ataque iminente [18].

O *Wireshark* foi identificado como um forte candidato para desenvolvimento adicional como um sistema fiável de deteção de intrusões, devido às suas várias opções de filtragem, incluindo filtragem por *substring*, tamanho de pacotes e protocolos [17].

Na Figura 2, observamos uma visão geral da interface gráfica de uma captura no *Wireshark*.

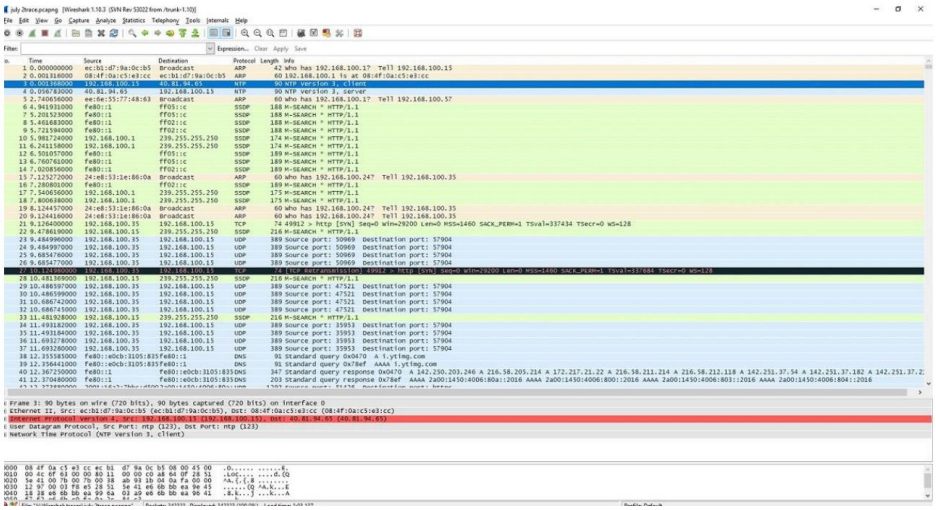


Figura 2 - Visão geral da interface gráfica de uma captura no *Wireshark [16]*.

2. Capturas

Nesta secção serão descritas as capturas de tráfego de rede propostas, fornecidas em arquivos *.pcap*, realizadas com a ferramenta *Wireshark*. Cada captura será analisada, detalhadamente, de forma a identificar os protocolos e as máquinas envolvidas, bem como, informações relevantes, como endereços IP e MAC. Procurou-se identificar atividades realizadas pelo utilizador, incluindo comandos e mensagens trocadas, e, sempre que possível, as credenciais

utilizadas. Para as capturas referentes a ataques (capturas 5 e 6), foram identificados o tipo de ataque, os objetivos e as vulnerabilidades exploradas, assim como os endereços das máquinas atacantes e dos alvos.

2.1 Captura 1

2.1.1 Protocolo Principal

Na análise da captura 1, verificamos a presença de alguns protocolos. Nas primeiras *frames* da captura, 1 e 2, está envolvido o protocolo ARP (*Adress Resolution Protocol*).

Mais tarde, na *frame* 6, é utilizado o protocolo HTTP, quando o cliente envia um pedido HTTP GET para o servidor. O servidor envia a resposta ao pedido, em pacotes TCP que são confirmados pelo cliente e finaliza a resposta com código "HTTP 200 OK", indicando que tudo foi transferido com sucesso.

2.1.2 Análise das Máquinas envolvidas na Captura

Nas primeiras *frames* da captura verificamos que estão envolvidas duas máquinas, uma correspondente ao cliente e outra ao servidor. A estrutura cliente-servidor é um modelo de arquitetura de rede amplamente utilizado em ambientes de TI. Nesta abordagem, os computadores são divididos em dois grupos: servidores, que fornecem serviços ou recursos, e clientes, que solicitam estes serviços e recursos, recebendo-os como respostas. A comunicação entre os dois é feita por meio de protocolos de rede, como TCP/IP, e é geralmente realizada pela internet ou por uma rede privada [19]. O cliente com IP 192.168.155.137 e endereço MAC 00:0c:29:ef:74:78 faz uma solicitação ARP em formato *broadcast* (para que todas as outras entidades na rede local ouçam a solicitação) para saber quem possui o endereço IP 192.168.155.181. Por sua vez, o dispositivo com endereço MAC 00:0c:29:3a:4c:1e responde, afirmando ser ele o dono do endereço IP procurado. Através destas *frames* obtemos de imediato os endereços IP e MAC das duas máquinas, como se verifica nas últimas linhas da figura 3, e percebemos que o cliente quer comunicar com o servidor, fazendo-o nas linhas seguintes através de pacotes TCP e do estabelecimento de conexão conhecido por *three-way handshake*, representada na figura 4.

```
▸ Frame 2: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth0, id 0
▸ Ethernet II, Src: VMware_3a:4c:1e (00:0c:29:3a:4c:1e), Dst: VMware_ef:74:78 (00:0c:29:ef:74:78)
▾ Address Resolution Protocol (reply)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: reply (2)
    Sender MAC address: VMware_3a:4c:1e (00:0c:29:3a:4c:1e)
    Sender IP address: 192.168.155.181
    Target MAC address: VMware_ef:74:78 (00:0c:29:ef:74:78)
    Target IP address: 192.168.155.137
```

Figura 3 - Painel de detalhes da captura 1.

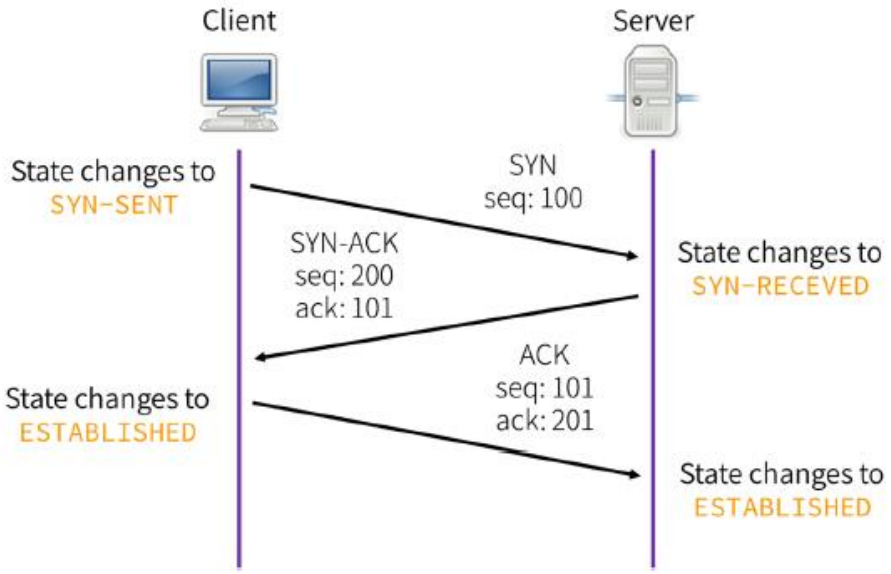


Figura 4 – Diagrama do TCP *Three-way Handshake [20]*.

No processo do *Three-Way Handshake* o cliente envia um pacote SYN (synchronize) para o servidor, que contém um número de sequência aleatório. Por sua vez, o servidor responde com um pacote SYN-ACK, contendo um número de sequência aleatório e um número de confirmação (ACK) que reconhece o número de sequência do cliente. Por fim, o cliente envia um número de confirmação (ACK) para o servidor, reconhecendo o número de sequência do servidor. Assim, os números de sequência em ambas as extremidades estão sincronizados e podem enviar e receber dados de forma independente [20]. Assim, após recolher as informações referentes a ambas as máquinas, esquematizou-se os endereços de cada máquina na tabela 1, abaixo representada.

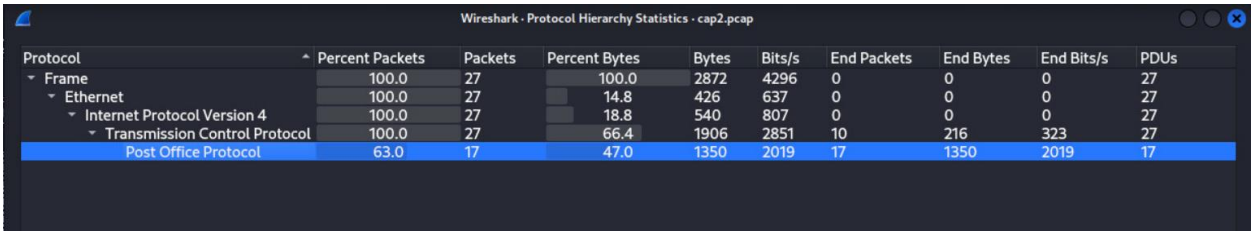
Tabela 1 – Informações referentes à *MAC address* e *IP adress* de ambas as máquinas envolvidas na captura 1.

	MAC *Address*	IP *Address*
Cliente	00:0c:29:ef:74:78, associado a um dispositivo identificado como VMwhare_ef:74:78	192.168.155.137
Servidor	00:0c:29:3a:4c:1e, associado a um dispositivo identificado como VMwhare_3a:4c:1e	192.168.155.181

2.2 Captura 2

2.2.1 Protocolo Principal

Ao analisar a captura 2 pode-se deduzir que o protocolo principal é o POP (*Post Office Protocol*), clicando em *Statistics* e de seguida, em *Protocol Hierarchy*, como demonstrado na figura 5.



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	27	100.0	2872	4296	0	0	0	27
Ethernet	100.0	27	14.8	426	637	0	0	0	27
Internet Protocol Version 4	100.0	27	18.8	540	807	0	0	0	27
Transmission Control Protocol	100.0	27	66.4	1906	2851	10	216	323	27
Post Office Protocol	63.0	17	47.0	1350	2019	17	1350	2019	17

Figura 5 -Visualização de estatísticas da hierarquia de protocolos da captura 2, no *Wireshark*.

2.2.2 Análise das Máquinas envolvidas na Captura

No âmbito da análise da segunda captura realizada no *Wireshark*, observa-se que o protocolo principal é o POP3 (*Post Office Protocol*) versão 3, observado anteriormente.

Entre as máquinas envolvidas encontram-se a correspondente ao cliente e ao servidor. Nas informações correspondentes à captura, (Figura 6) a seção *Ethernet II* exibe os endereços MAC da fonte e destino para a captura analisada. Por outro lado, os detalhes referentes as portas do cliente (*source port*) e do servidor (*destination port*) encontram-se na secção *Transmission Control Protocol*. A porta 110 é utilizado como porta de origem para o servidor POP3. A porta de destino é a 1140, utilizada pelo cliente para estabelecer a sessão de comunicação específica com o servidor.

```
▾ Ethernet II, Src: SanyoDenki_10:f9:4c (00:e0:4e:10:f9:4c), Dst: RealtekSemic_16:c8:94 (00:e0:4c:16:c8:94)
  ▸ Destination: RealtekSemic_16:c8:94 (00:e0:4c:16:c8:94)
  ▸ Source: SanyoDenki_10:f9:4c (00:e0:4e:10:f9:4c)
    Type: IPv4 (0x0800)
▸ Internet Protocol Version 4, Src: 64.246.26.20, Dst: 192.168.2.100
▾ Transmission Control Protocol, Src Port: 110, Dst Port: 1140, Seq: 233, Ack: 67, Len: 1003
    Source Port: 110
    Destination Port: 1140
```

Figura 6 - Painel de detalhes do pacote da captura 2.

As informações referentes a ambas as máquinas encontram-se esquematizadas na tabela 2, abaixo representada.

Tabela 2 - Informações referentes à *MAC address* e *IP adress* de ambas as máquinas envolvidas na captura 2.

	MAC Address	*IP Address*
Cliente	00:e0:4c:16:c8:94, associado a um dispositivo identificado como RealtekSemic_16:c8:94	192.168.2.100
Servidor	00:e0:4e:10:f9:4c, associado a um dispositivo identificado como SanyoDenki_10:f9:4c	64.246.26.20

2.2.3 Comandos e Funcionalidades solicitadas

Nas primeiras três *frames* da captura, pode-se observar que o cliente (192.168.2.100) inicia uma conexão do tipo TCP com o servidor (64.246.26.20) na porta 110, utilizando o processo *three-way handshake* (SYN, SYN-ACK, ACK) (Figura 7). Este processo permite a sequência de troca de pacotes durante a conexão TCP. Primeiramente, o cliente inicia uma comunicação, enviando um pacote SYN (*synchronize*) ao servidor. Este pacote permite que o cliente informe o servidor que deseja abrir uma conexão e que está pronto para receber dados. Ao receber o pacote SYN, o servidor responde com um pacote SYN-ACK, de forma a confirmar que recebeu o pedido para iniciar a conexão. Por fim, o pacote ACK (*Acknowledgment*) é enviado do cliente para o servidor, contendo o número de reconhecimento, de forma a confirmar que recebeu o pacote anterior.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.2.100	64.246.26.20	TCP	62	1140 → 110 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM
2	0.464000	64.246.26.20	192.168.2.100	TCP	62	110 → 1140 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1452 SACK_PERM
3	0.464000	192.168.2.100	64.246.26.20	TCP	60	1140 → 110 [ACK] Seq=1 Ack=1 Win=65535 Len=0

Figura 7 - *Three Way Handshake* entre o cliente e servidor.

Por fim, o cliente e o servidor estão sincronizados e a conexão TCP é estabelecida entre o servidor e o cliente, informando que o servidor “mail.colasoft.com” se encontra pronto para

receber comandos (*frame* 4). De seguida, é realizada a autenticação do utilizador (figura 8), sendo o seu nome do utilizador "test2@colasoft.com". O servidor, por sua vez, envia um pacote a reconhecer que recebeu a informação, nomeadamente a *flag* ACK, aceitando a credencial e solicitando a palavra-passe. A palavra-passe introduzida é "test2123", sendo novamente reconhecida pelo servidor.

5	0.937000	192.168.2.100	64.246.26.20	POP	79 C: USER test2@colasoft.com
6	1.395000	64.246.26.20	192.168.2.100	TCP	60 110 → 1140 [ACK] Seq=53 Ack=26 Win=5840 Len=0
7	1.397000	64.246.26.20	192.168.2.100	POP	95 S: +OK User name accepted, password please
8	1.401000	192.168.2.100	64.246.26.20	POP	69 C: PASS test2123

Figura 8 - Inserção das credenciais de autenticação por parte do cliente.

Após a autenticação, é realizado o acesso à caixa de correio do email, onde foram realizados comandos, durante a sessão. O cliente verificou que possui uma mensagem por abrir (*frame* 10).

Seguidamente, o cliente realiza o comando STAT. Este permite obter informações detalhadas de um arquivo ou diretório, como tipo de ficheiro, permissão de acesso, data de acesso/modificação/criação do arquivo, entre outras [21]. Desta forma, o servidor responde com a *flag* ACK e envia a informação que existe uma mensagem nova na caixa de correio do email, com 984 bytes, como se pode observar na figura 9.

11	1.963000	192.168.2.100	64.246.26.20	POP	60 C: STAT
12	2.465000	64.246.26.20	192.168.2.100	TCP	60 110 → 1140 [ACK] Seq=124 Ack=47 Win=5840 Len=0
13	2.465000	64.246.26.20	192.168.2.100	POP	65 S: +OK 1 984

Figura 9 - Aplicação do comando STAT.

O comando UIDL (*Unique-ID Listing*), por sua vez, trata-se de uma lista de diretórios em POP3 (Figura 10). Quando um cliente emite esta solicitação, o servidor responde com uma lista de *strings* de "id exclusivo", podendo estas ser considerados nomes de arquivo. Cada ID exclusivo é emparelhado com um ID numérico, começando por 1. Torna-se importante reiterar que todos os outros comandos para baixar e excluir mensagens usam esses IDs numéricos [22]. No contexto da captura estudada, o servidor responde com a identificação da mensagem, como se pode observar pela figura 11.

14	2.466000	192.168.2.100	64.246.26.20	POP	60 C: UIDL
15	2.920000	64.246.26.20	192.168.2.100	POP	108 S: +OK Unique-ID listing follows

Figura 10 - Aplicação do comando UIDL.

```
▾ Post Office Protocol
  ▸ +OK Unique-ID listing follows\r\n
    1 4224ff1c00000001\r\n
    .\r\n
```

Figura 11 - Retorno do servidor ao comando UIDL.

Finalmente, o cliente emite o comando LIST (Figura 12), fornecendo ao servidor uma solicitação de detalhes da mensagem. Este comando tem como intuito listar todas as mensagens e retorna uma lista indexada das mesmas, juntamente com o tamanho [23]. O servidor retorna com "+OK", exibindo a mensagem que o cliente possui no e-mail e indicando a quantidade de bytes correspondente à mesma — 98 bytes (Figura 13).

16	2.920000	192.168.2.100	64.246.26.20	POP	60 C: LIST
17	3.497000	64.246.26.20	192.168.2.100	POP	98 S: +OK Mailbox scan listing follows

Figura 12 - Aplicação do comando LIST.

```
▸ Frame 17: 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
▸ Ethernet II, Src: SanyoDenki_10:f9:4c (00:e0:4c:10:f9:4c), Dst: RealtekSemic_16:c8:94 (00:e0:4c:16:c8:94)
▸ Internet Protocol Version 4, Src: 64.246.26.20, Dst: 192.168.2.100
▸ Transmission Control Protocol, Src Port: 110, Dst Port: 1140, Seq: 189, Ack: 59, Len: 44
▾ Post Office Protocol
  ▾ +OK Mailbox scan listing follows\r\n
      Response indicator: +OK
      Response description: Mailbox scan listing follows
    1 984\r\n
    .\r\n
```

Figura 13 - Retorno do servidor ao comando LIST.

Nas *frames* 18 e 19, presentes na figura 14, é possível observar que foi utilizado o comando RETR, de forma a recuperar o conteúdo da primeira mensagem de e-mail [23]. O servidor responde com "+OK 984 octetos", indicando que a mensagem tem 984 bytes.

18	3.499000	192.168.2.100	64.246.26.20	POP	62 C: RETR 1
19	3.959000	64.246.26.20	192.168.2.100	POP	1057 S: +OK 984 octets

Figura 14 - Aplicação do comando RETR.

Na figura 15, por sua vez, pode-se verificar que, no *Post Office Protocol* encontram-se as informações recolhidas aquando da aplicação do comando RETR. Assim, pode-se verificar os detalhes do email, nomeadamente o remetente (test1@colasoft.com) e destinatário (test2@colasoft.com) e o conteúdo da mensagem recebida.

```
▾ Post Office Protocol
  ▾ +OK 984 octets\r\n
      Response indicator: +OK
      Response description: 984 octets
    Return-Path: <test1@colasoft.com>\r\n
    Received: from mail.colasoft.com (Unknown UID 8@localhost)\r\n
    \tby colasoft.com (8.11.6/8.11.6) with ESMTP id j21NjO300738\r\n
    \tfor <test2@colasoft.com>; Tue, 1 Mar 2005 17:45:24 -0600\r\n
    X-Authentication-Warning: mail.colasoft.com: Unknown UID 8 owned process doing -bs\r\n
    X-ClientAddr: 222.210.195.15\r\n
    Received: from colasoft_test_1 ([222.210.195.15])\r\n
    \t(authenticated (0 bits))\r\n
    \tby mail.colasoft.com (8.11.6/8.11.6) with ESMTP id j21NjMd00731\r\n
    \tfor <test2@colasoft.com>; Tue, 1 Mar 2005 17:45:23 -0600\r\n
    Message-Id: <200503012345.j21NjMd00731@mail.colasoft.com>\r\n
    Date: Wed, 2 Mar 2005 10:19:39 -0800\r\n
    From: "test1@colasoft.com" <test1@colasoft.com>\r\n
    To: "test2" <test2@colasoft.com>\r\n
    Subject: test email!\r\n
    X-mailer: Foxmail 5.0 [en]\r\n
  Mime-Version: 1.0\r\n
  Content-Type: text/plain;\r\n
  \tcharset="iso-8859-1"\r\n
  Content-Transfer-Encoding: 7bit\r\n
  Status:   \r\n
  \r\n
  Hello test2,\r\n
  \r\n
          this email is test!\r\n
  \r\n
  Best regards, \r\n
    \r\n
  test1\r\n
```

Figura 15 - Retorno do servidor ao comando RETR.

Nas *frames* 20 e 21, explícitas na figura 16, o cliente emite o comando DELE, de modo a excluir a mensagem especificada [23]. O servidor, por sua vez, responde com "+OK Message deleted", confirmando que a mesma foi marcada para exclusão.

20	3.967000	192.168.2.100	64.246.26.20	POP	62	C: DELE 1
21	4.422000	64.246.26.20	192.168.2.100	POP	75	S: +OK Message deleted

Figura 16 - Aplicação do comando DELE.

Posteriormente, o cliente envia o comando QUIT, indicando que pretende terminar a sessão e guardar quaisquer alterações [23]. O servidor responde com "+OK Sayonara", confirmando o fim da sessão (*frames* 22 e 23 – Figura 17). Após o comando QUIT, a conexão TCP é fechada através de uma série de pacotes FIN, ACK, de modo que tanto o cliente quanto o servidor fecham a conexão.

22	4.424000	192.168.2.100	64.246.26.20	POP	60	C: QUIT
23	4.889000	64.246.26.20	192.168.2.100	POP	68	S: +OK Sayonara
24	4.889000	192.168.2.100	64.246.26.20	TCP	60	1140 → 110 [FIN, ACK] Seq=81 Ack=1271 Win=64265 Len=0
25	4.889000	64.246.26.20	192.168.2.100	TCP	60	110 → 1140 [FIN, ACK] Seq=1271 Ack=81 Win=5840 Len=0
26	4.889000	192.168.2.100	64.246.26.20	TCP	60	1140 → 110 [ACK] Seq=82 Ack=1272 Win=64265 Len=0
27	5.347000	64.246.26.20	192.168.2.100	TCP	60	110 → 1140 [ACK] Seq=1272 Ack=82 Win=5840 Len=0

Figura 17 - Aplicação dos comandos QUIT e finalização da comunicação entre o cliente e o servidor.

2.3 Captura 3

2.3.1 Protocolo Principal

Através da análise da captura 3 é possível verificar que a mesma apresenta 7884 *frames* (figura 18).

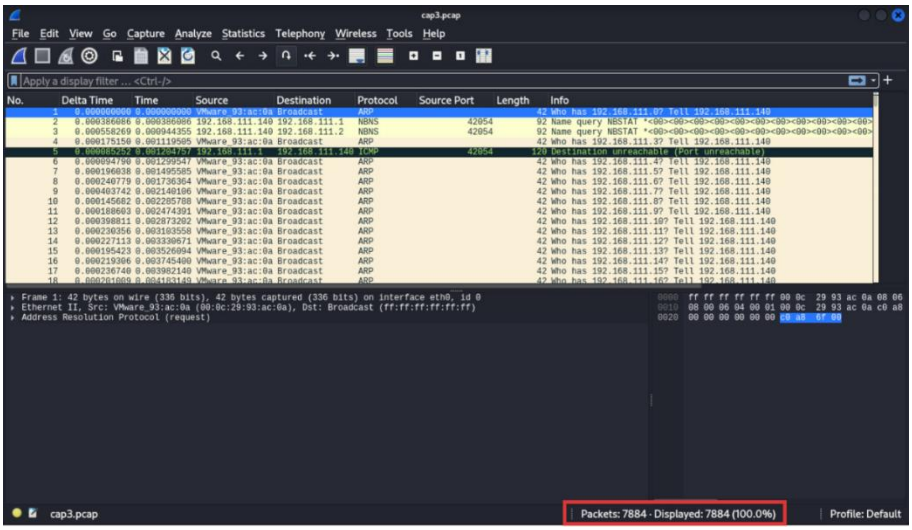


Figura 18 - Exemplo ilustrativo da quantidade de *frames* associado à captura 3.

Ao clicar em *Statistics*, e depois em *Protocol Hierachy* é possível aceder-se a uma ferramenta estatística que mostra, de forma hierárquica, todos os protocolos envolvidos na captura, como é possível verificar na figura 19.

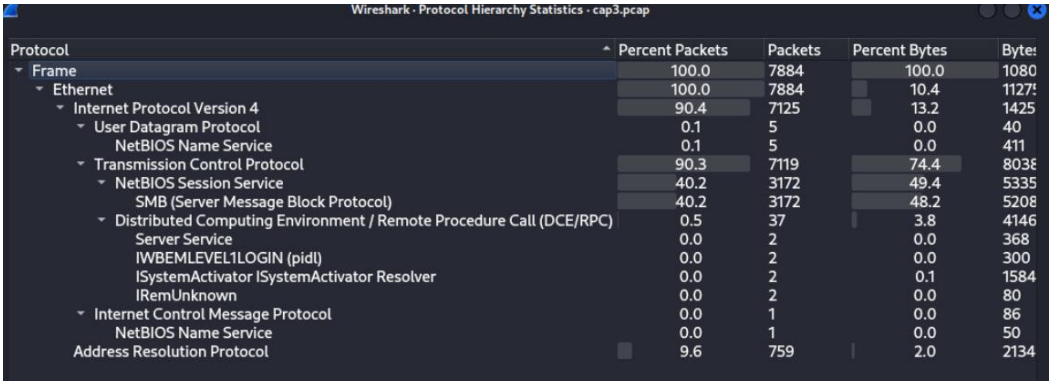


Figura 19 - Visualização de estatísticas da hierarquia de protocolos da captura 3, no *Wireshark*.

Analisando a figura 19, anteriormente apresentada, conclui-se que os principais protocolos envolvidos nesta captura são o SMB (*Server Message Block*) e o TCP (*Transmission Control Protocol)*. Para além destes dois protocolos, existem ainda nesta captura um conjunto de outros

protocolos de suporte, como o ARP (*Address Resolution Protocol*), o DCE/RPC (*Distributed Computing Environment / Remote Procedure Call*) e o UDP (*User Datagram Protocol*).

2.3.2 Análise das Máquinas envolvidas na Captura

As figuras 20 e 21 apresentam as principais máquinas envolvidas nesta captura, que confirmam os dados presentes na Tabela 3.

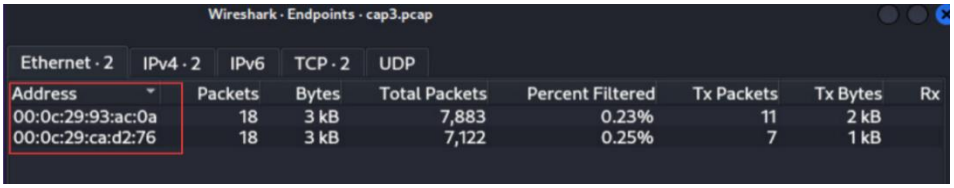


Figura 20 - Endereços MAC das duas máquinas envolvidas na captura 3.

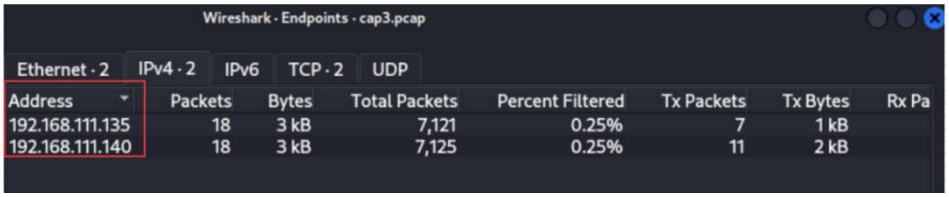


Figura 21 - Endereços de IP das duas máquinas envolvidas na captura 3.

Por outro lado, a figura 22 apresenta os endereços de IP de todas as "conversas" que existiram ao longo da captura. É de notar a predominância de *packets* entre as máquinas de IP: 192.168.111.140 e 192.168.111.135, que correspondem às máquinas anteriormente apresentadas e que serão consideras as principais máquinas desta captura.

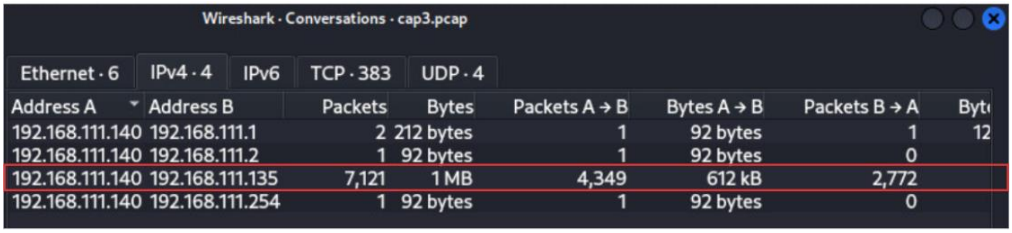


Figura 22 - IPs e quantidade de *packets* envolvidos em todas as conversas existentes na captura 3.

Na Tabela 3, estão representados os endereços MAC e IP das máquinas presentes na captura 3.

Tabela 3 - Informações referentes à MAC *address* e IP *adress* de ambas as máquinas envolvidas na captura 3.

	MAC *Address*	**IP *Address***
Cliente	VMware_93:ac:0a (00:0c:29:93:ac:0a)	192.168.111.140
Servidor	VMware_ca:d2:76 (00:0c:29:ca:d2:76)	192.168.111.135

2.3.3 Comandos e Funcionalidades solicitadas

Analisando as primeiras 5 *frames* desta captura (figura 23), verifica-se que a mesma se inicia com requisições do tipo ARP, um protocolo que é utilizado para resolver endereços IP para endereços MAC. A máquina de origem (*sender*) com IP: 192.168.111.140 e MAC *adress*: 00:0c:29:93:ac:0a está a tentar descobrir o endereço MAC da máquina de destino (*target*).

No.	Source	Destination	Protocol	Length	Source Port	Destination Port	Info
1	VMware_93:ac:0a	Broadcast	ARP	42			Who has 192.168.111.0? Tell 192.168.111.140
2	192.168.111.140	192.168.111.1	NBNS	92	42054	137	Name query NBSTAT *<00><00><00><00><00><00><00><00><00><00><00><00><00><00><00>
3	192.168.111.140	192.168.111.2	NBNS	92	42054	137	Name query NBSTAT *<00><00><00><00><00><00><00><00><00><00><00><00><00><00><00>
4	VMware_93:ac:0a	Broadcast	ARP	42			Who has 192.168.111.3? Tell 192.168.111.140
5	192.168.111.1	192.168.111.140	ICMP	120			Destination unreachable (Port unreachable)

Figura 23 - Painel de visualização da captura 3, das *frames* 1-5.

Na *frame 1* (figura 24) o cliente enviou uma requisição ARP em *Broadcast* para descobrir o endereço MAC corresponde ao IP 192.168.111.0. Este processo faz parte da resolução ARP, onde a máquina tenta mapear o IP de destino para um MAC, verificando assim se existe alguma máquina na rede com esse IP.

```
▸ Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface eth0, id 0
▾ Ethernet II, Src: VMware_93:ac:0a (00:0c:29:93:ac:0a), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  ▾ Destination: Broadcast (ff:ff:ff:ff:ff:ff)
      .... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
      .... ...1 .... .... .... .... = IG bit: Group address (multicast/broadcast)
  ▾ Source: VMware_93:ac:0a (00:0c:29:93:ac:0a)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: ARP (0x0806)
    [Stream index: 0]
▾ Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: VMware_93:ac:0a (00:0c:29:93:ac:0a)
    Sender IP address: 192.168.111.140
    Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
    Target IP address: 192.168.111.0
```

Figura 24 - Painel de detalhes do pacote da captura 3, da *frame* 1.

Da totalidade de 759 *frames* de requisições ARP (aplicação do filtro "arp"), verifica-se que 755 *frames* (aplicação do filtro "*arp.opcode==1*") dizem respeito a requisições ARP para *Broadcast*, e apenas 4 *frames* (filtro "*arp.opcode==2*") são respostas ARP. Não obstante, todas estas solicitações do tipo ARP são realizadas com o intuito e determinar o MAC *address* para IPs específicos.

Nas *frames* 2 e 3 ocorre uma tentativa de comunicação entre duas máquinas (192.168.111.140 para a máquina 192.168.111.1, no caso da *frame* 2) através do protocolo UDP, onde a consulta é realizada através do serviço *NetBIOS Name Service* (NBNS) (porta 137), que é um serviço utilizado para resolução de nomes na rede local, especialmente em redes *Windows* [24].

```
NetBIOS Name Service
    Transaction ID: 0x0393
  ▾ Flags: 0x0010, Opcode: Name query, Broadcast
        0... .... .... .... = Response: Message is a query
        .000 0... .... .... = Opcode: Name query (0)
        .... ..0. .... .... = Truncated: Message is not truncated
        .... ...0 .... .... = Recursion desired: Don't do query recursively
        .... .... ...1 .... = Broadcast: Broadcast packet
```

Figura 25 - Painel de detalhes do pacote da captura 3, da *frame* 2.

A consulta foi enviada em *broadcast* (Figura 25), com o objetivo de descobrir o *status* de nomes NetBIOS na rede.

Relativamente à *frame 5*, esta consiste numa resposta à *frame 2*. É utilizado o protocolo ICMP *(Internet Control Message Protcol)* com o código 3 ("*Port Unreachable"*), indicando que a máquina não pode responder à solicitação na porta 137 (destino), por não estar aberta ou disponível para comunicação.

Da *frame 5* à *frame* 136 verifica-se a existência de apenas solicitações ARP semelhantes às requisições anteriormente explicadas. A única diferença centra-se apenas na variação do IP de destino nas requisições ARP, que irá variar até que algum dispositivo de destino com o endereço IP em questão responda diretamente com um pacote ARP de resposta, mapeando-se assim o endereço MAC de destino. Portanto, a existência de uma grande quantidade de requisições ARP não constitui um erro, uma vez que fazem parte do processo normal de resolução de endereços na rede.

Na *frame 137* volta a ocorrer uma consulta NetBIOS do tipo NBSTAT enviado via UDP para a porta 137 (que é reservada para NBNS), onde a máquina de origem tem um endereço IP:

192.169.111.140 e a máquina de destino: 192.168.111.135. Existe, portanto, uma tentativa de descobrir ou confirmar o estado de um nome NetBIOS do destino, que irá servir como um identificador da rede local, facilitando a comunicação entre computadores *Windows*. A resposta à consulta NBNS feita nesta *frame* encontra-se na *frame* 141, sendo possível correlacionar esta resposta com a consulta original através do *Transaction ID* associado a ambas as solicitações, que neste caso é 0x03a7 (figura 26). Para além disso, esta inclui uma lista de nomes NetBIOS associadas à máquina cujo endereço IP é 192.168.111.135.

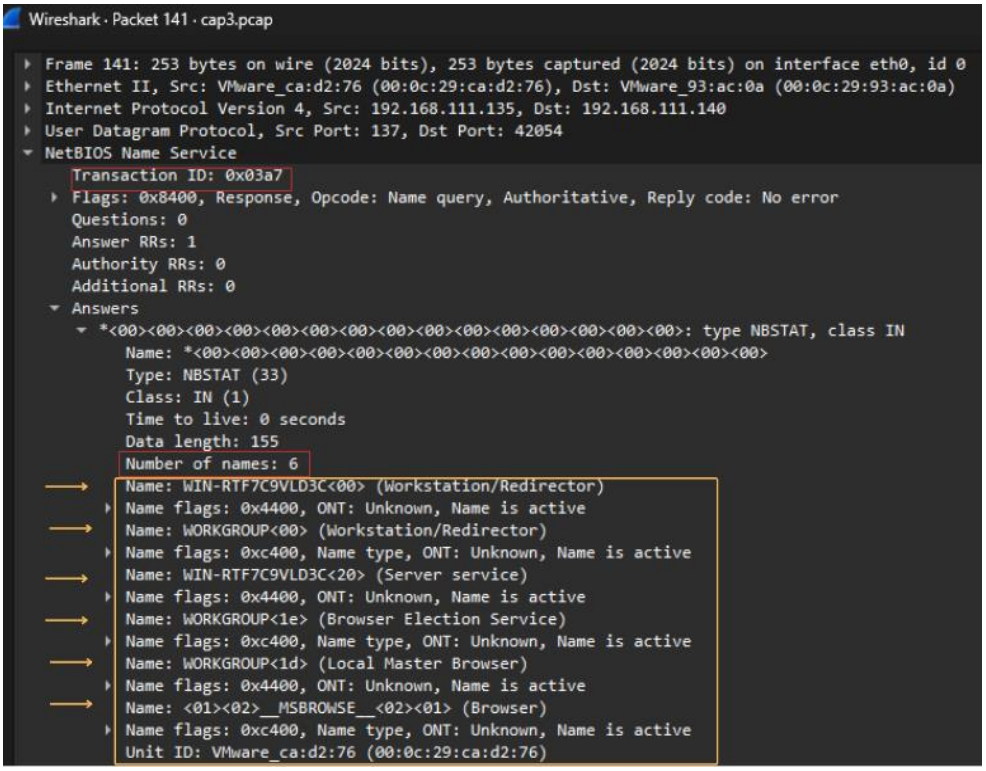


Figura 26 - Resposta NBNS ao pedido de nomes NetBIOS realizado na *frame* 137.

Relativamente à lista de nomes obtidos pela consulta NetBIOS (sinalizados com uma seta na figura 26), podem-se tirar as seguintes conclusões [25]:

- **WIN-RTF7C9VLD3C<00>** - Nome de identificação do dispositivo na rede, normalmente representa o nome do sistema ou nome de *host*;
- **WORKGROUP<00>** - Nome do grupo de trabalho a que o dispositivo pertence, utilizado para organizar dispositivos numa rede local;

- **WIN-RTF7C9VLD3C<20> -** Indica a presença de um serviço de servidor (*Server Service*) no dispositivo (identificado pelo sufixo <20>);
- **WORKGROUP<1e>** - Indica o serviço de eleição de navegação (*Browser Election Service*). Este serve para decidir qual dispositivo irá atuar como navegador mestre no grupo de trabalho e listará os dispositivos disponíveis;
- **WORKGROUP<1d>** - Navegador Mestre Local (*Local Master Browser*), responsável por manter uma lista atualizada de dispositivos ativos no grupo de trabalho;
- **<01><02>MSBROWSE<02><01>** - Este nome de serviço é reservado para o navegador mestre, e identifica a máquina responsável pela listagem e monitoramento da rede.

Nas *frames* seguintes, até à 645, verifica-se a ocorrência de apenas solicitações do tipo ARP e algumas consultas pontuais de NBNS. Esta predominância de apenas estes dois protocolos pode ser indício do início de um possível ataque, sendo esta fase categorizada como "*Reconnaissance* – fase de reconhecimento", onde o objetivo centra-se com o alegado atacante a tentar mapear e entender a rede antes de aprofundar o seu ataque. Portanto, os *ARP Requests* compõem um método de descoberta dos dispositivos da rede, onde o atacante tenta descobrir os seus possíveis "alvos" e respetivos endereços de IP. Por outro lado, as solicitações NBNS são utilizadas para obter informações mais concretas dos dispositivos da rede, incluindo os seus nomes de *host* e grupos de trabalho, o que permite que o ataque descubra quais máquinas estão disponíveis e quais serviços estão em execução.

A partir da *frame 646*, inicia-se um fluxo TCP (*TCP Stream*), que segue um padrão repetitivo até à *frame 7552*, momento no qual esse padrão é interrompido, verificando-se um comportamento diferente do até então observado. A seguir, será explicado em que é que consiste esse *TCP Stream*, analisando o bloco de *packets*/*frames* que se repete ao longo da maior parte da captura. E, posteriormente, será feita uma análise aos acontecimentos que levaram à quebra desse padrão e das ações subsequentes.

A análise do fluxo TCP que se inicia na *frame 646* pode ser feita através da aplicação do filtro: "*tcp.stream eq 0"* (Figura 27).

tcp.stream eq 0

No.	DeltaTime	Time	Source	Destination	Protocol	Length	Source Port	Destination Port	Info
646	0.332120983	4.467174380	192.168.111.140	192.168.111.135	TCP	74	41405	445	41405 → 445 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1240538935 TSecr=0 WS=128
647	0.000438848	4.467613228	192.168.111.135	192.168.111.140	TCP	74	445	41405	445 → 41405 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM TSval=34932 TSecr=1240538935
648	0.000036935	4.467650163	192.168.111.140	192.168.111.135	TCP	66	41405	445	41405 → 445 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1240538936 TSecr=34932
649	0.002065353	4.469715516	192.168.111.140	192.168.111.135	SMB	139	41405	445	Negotiate Protocol Request
650	0.000824815	4.470540331	192.168.111.135	192.168.111.140	SMB	197	445	41405	Negotiate Protocol Response
651	0.000032525	4.470572856	192.168.111.140	192.168.111.135	TCP	66	41405	445	41405 → 445 [ACK] Seq=74 Ack=132 Win=64128 Len=0 TSval=1240538939 TSecr=34932
652	0.002614550	4.473187406	192.168.111.140	192.168.111.135	SMB	273	41405	445	Session Setup AndX Request, NTLMSSP_NEGOTIATE
654	0.000356342	4.473891870	192.168.111.135	192.168.111.140	SMB	417	445	41405	Session Setup AndX Response, NTLMSSP_CHALLENGE, Error: STATUS_MORE_PROCESSING_REQUIRED
655	0.000054988	4.473946858	192.168.111.140	192.168.111.135	TCP	66	41405	445	41405 → 445 [ACK] Seq=281 Ack=483 Win=64128 Len=0 TSval=1240538942 TSecr=34932
656	0.010494715	4.484441573	192.168.111.140	192.168.111.135	SMB	547	41405	445	Session Setup AndX Request, NTLMSSP_AUTH, User: .\john
657	0.000941123	4.485382696	192.168.111.135	192.168.111.140	SMB	105	445	41405	Session Setup AndX Response, Error: STATUS_LOGON_FAILURE
658	0.000021943	4.485404639	192.168.111.140	192.168.111.135	TCP	66	41405	445	41405 → 445 [ACK] Seq=762 Ack=522 Win=64128 Len=0 TSval=1240538954 TSecr=34933
659	0.003104774	4.488509413	192.168.111.140	192.168.111.135	SMB	109	41405	445	Logoff AndX Request
660	0.000404727	4.488914140	192.168.111.135	192.168.111.140	SMB	105	445	41405	Logoff AndX Response, Error: Bad userid
661	0.000013505	4.488927645	192.168.111.140	192.168.111.135	TCP	66	41405	445	41405 → 445 [ACK] Seq=805 Ack=561 Win=64128 Len=0 TSval=1240538957 TSecr=34934
662	0.002098601	4.491026246	192.168.111.140	192.168.111.135	TCP	66	41405	445	41405 → 445 [FIN, ACK] Seq=805 Ack=561 Win=64128 Len=0 TSval=1240538959 TSecr=34934
663	0.000344224	4.491370470	192.168.111.135	192.168.111.140	TCP	66	445	41405	445 → 41405 [ACK] Seq=561 Ack=806 Win=65792 Len=0 TSval=34934 TSecr=1240538959
664	0.000019331	4.491389801	192.168.111.135	192.168.111.140	TCP	60	445	41405	445 → 41405 [RST, ACK] Seq=561 Ack=806 Win=0 Len=0

Figura 27 - *TCP Stream* que se repete ao longo da captura até à *frame* 7552. Aplicação do filtro "*tcp.stream eq 0*".

O principal protocolo envolvido neste *stream* é o TCP, estando a ser utilizado para estabelecer uma conexão com a porta 445 do dispositivo de destino (IP: 192.168.111.135 e MAC: 00:0c:29:ca:d2:76). Normalmente, a porta 445 está associada ao *Microsoft-DS* (*Directory Services)*, utilizada para o serviço SMB**,** que por sua vez é utilizado para compartilhar arquivos em redes *Windows* e também executar comando remotos, como já visto anteriormente. Para além disso, o protocolo SMB pode também ser utilizado para tentativas de exploração de vulnerabilidades.

Na *frame* 646 verifica-se o início do estabelecimento da conexão TCP, onde é enviada uma *flag* SYN, que é o primeiro passo do protocolo *Three-Way-Handshake* do TCP. O segundo passo deste protocolo surge na *frame* seguinte (*frame 647*) com as *flags* SYN e ACK ativadas, o que indica que este *packet* é uma resposta ao pedido de conexão feito anteriormente onde o SYN foi enviado, e que o servidor está pronto para aceitar a conexão. A conexão TCP é concluída com o cliente, na *frame* 648, a responder com uma *flag* ACK, o que confirma que este recebeu o SYN-ACK e que está pronto para continuar com a comunicação, terminando, assim, o processo de três vias de conexão TCP.

Uma vez estabelecida a conexão TCP, a comunicação segue com uma solicitação, por parte do cliente para o servidor, de uma negociação do protocolo de comunicação SMB (*frame 649*), verificado pelo comando *Negotiate Protocol* (0x72). Nesta solicitação estão presentes as *flags* PSH e ACK (*Push* e *Acknowledgment*), sendo a PSH um indicativo de que o TCP está a solicitar a entrega imediata de dados. Para além disso, verifica-se a utilização do NetBIOS na comunicação SMB, onde é carregada uma mensagem de sessão (*Session Message*) que contém uma negociação

de protocolo com os dialetos SMB (versões ou opções de SMB) que o cliente está disposto a utilizar (Figura 28). Esta etapa de negociação de protocolo é fundamental para que o cliente e o servidor determinem qual versão do SMB será utilizada na comunicação, do modo a haver compatibilidade em ambos os lados.

```
▾ Negotiate Protocol Request (0x72)
    Word Count (WCT): 0
    Byte Count (BCC): 34
  ▾ Requested Dialects
    ▾ Dialect: NT LM 0.12
        Buffer Format: Dialect (2)
        Name: NT LM 0.12
    ▾ Dialect: SMB 2.002
        Buffer Format: Dialect (2)
        Name: SMB 2.002
    ▾ Dialect: SMB 2.???
        Buffer Format: Dialect (2)
        Name: SMB 2.???
```

Figura 28 - Dialetos SMB oferecidos pelo cliente para o servidor escolher.

A resposta a esta solicitação de negociação de protocolo apresentada anteriormente na *frame 649*, encontra-se na *frame 650*, onde o servidor responde com a confirmação dos parâmetros e versão do SMB que será utilizada. Novamente estão presentes as *flags* PSH e ACK, onde indicam que o servidor está a confirmar a receção da solicitação negociação e quer que a mensagem deste *packet* seja entregue/processada imediatamente pelo cliente. No *Session Message* encontra-se a resposta SMB do servidor, onde a escolha para a comunicação é indicada no campo *Selected Index*: 0: NT LM 0.12, também conhecida por SMB1, que é uma versão mais antiga do protocolo SMB e amplamente suportada e compatível. O servidor responde também com o código *STATUS_SUCESS* para indicar que a negociação foi concluída com sucesso. Para além disso, são ainda definidos pelo servidor os modos e capacidade de segurança, nomeadamente o *USER security mode* que está associado a senhas criptografadas (Figura 29).

```
▾ SMB (Server Message Block Protocol)
  ▸ SMB Header
  ▾ Negotiate Protocol Response (0x72)
      Word Count (WCT): 17
      Selected Index: 0: NT LM 0.12
    ▾ Security Mode: 0x03, Mode, Password
        .... ...1 = Mode: USER security mode
        .... ..1. = Password: ENCRYPTED password. Use challenge/response
        .... .0.. = Signatures: Security signatures NOT enabled
        .... 0... = Sig Req: Security signatures NOT required
```

Figura 29 - Definição dos modos de segurança do servidor.

Posteriormente, já na *frame* 651, verifica-se a confirmação do cliente da versão do SMB proposta pelo servidor e, com a negociação concluída, dá-se início à fase de autenticação e/ou troca de dados. A *frame* 652 mostra o processo de negociação e autenticação NTLM em andamento, onde o comando SBM utilizado é o *Session Setup AndX* (0x73), o que indica que o cliente está a tentar estabelecer uma sessão autenticada com o servidor. Existe ainda o envio de um *Security Blob* que contém informações sobre o protocolo de autenticação escolhido (SPNEGO), o qual irá negociar o método de autenticação adequado, tendo sido escolhido o NTLMSSP (*NT LAN Manager Security Support Provider*) (Figura 30).

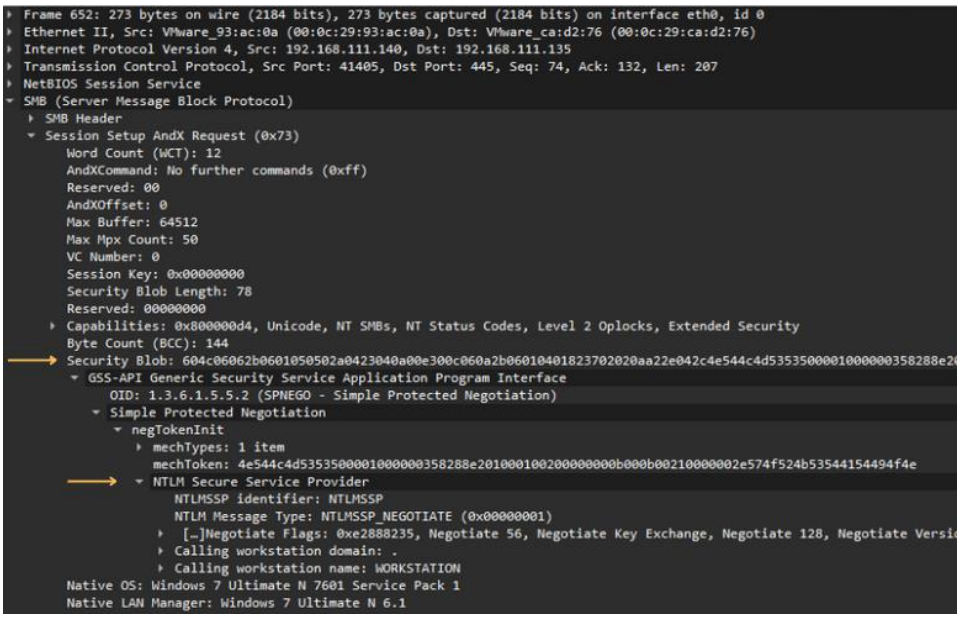


Figura 30 - Painel de detalhes do pacote da captura 3, da *frame* 652.

A resposta do servidor à solicitação anterior (*NTLMSSP_NEGOTIATE*) está presente na *frame 654*, onde é fornecido um desafio ao cliente (*NTLMSSP_CHALLENGE*), necessário para a dar continuidade ao processo de autenticação. Para além disso, o código de *status STATUS_MORE_PROCESSING_REQUIRED (0xc0000016)* sugere que o servidor indica que o cliente necessita de continuar o processo de autenticação. Através da análise de alguns detalhes da *frame* 654 (Figura 31) sabe-se que o nome do alvo (*Target Name*) é WIN-RTF7C9VLD3C, que é o nome de *host* do servidor que está a desafiar o cliente, como também que o sistema operacional do servidor é o *Windows* 8.1 9600.

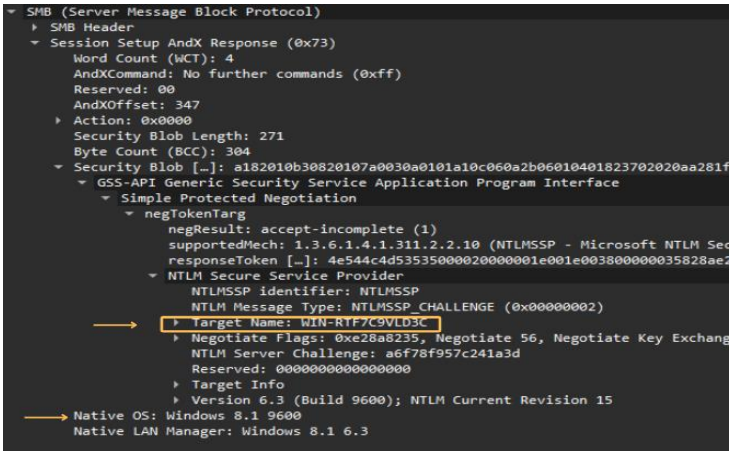


Figura 31 - Painel de detalhes do pacote da captura 3, da *frame* 654.

Na *frame* 655 surge um pacote TCP com uma *flag* ACK, este pacote atua como pacote de controle, ou seja, ele irá permitir a continuidade e estabilidade da conexão, garantido a ordem e confirmação do recebimento correto das sequências anteriores.

O pacote seguinte, referente à *frame 656* representa um pacote de autenticação SMS/NTLMv2 enviado pelo cliente para o servidor, correspondendo a uma tentativa de estabelecer uma sessão autenticada. Volta a ser utilizado o comando *SBM Session Setup AndX* (0x73) como forma de *Request*. Esta *frame* inclui um conjunto de campos (Figura 32) como NTLMv2, *NTLMSSP_AUTH*, resposta *Lan Manager*, que indicam que o cliente está a enviar credenciais para validação de identidade. As credenciais de *password* não são visíveis diretamente nos dados apresentado, uma vez que estas são transmitidas de forma criptografada no protocolo NTLM. Assim, o cliente enviar uma resposta NTLMv2, que consiste num *hash* da *password* combinado com outros dados, sendo assim necessário realizar um ataque de força bruta para descobrir o seu contéudo. Não obstante, neste pacote é possível encontrar alguns detalhes sobre o cliente, especificamente o nome de utilizador: John, bem como o nome do *host*: WORKSTATION, e o sistema operacional do mesmo*: Windows 7 Ultimate N 7601 Service Pack 1*.

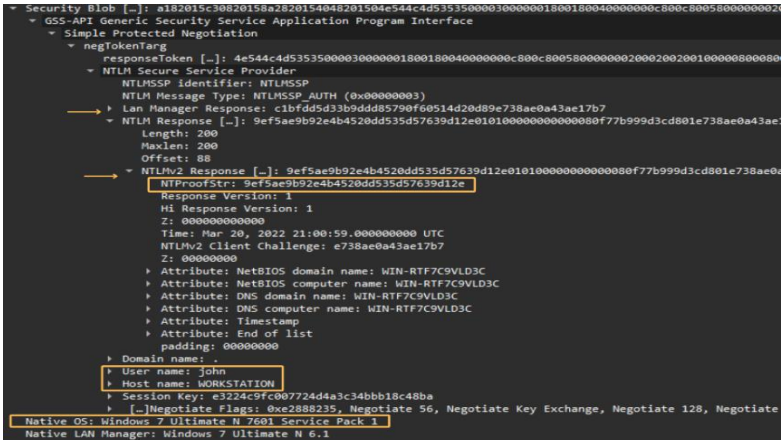


Figura 32 - Painel de detalhes do pacote da captura 3, da *frame* 656.

A resposta do servidor SMB para o cliente, referente ao pacote da *frame* 656, encontra-se na *frame* 657 onde é informada a ocorrência de uma falha de autenticação (*logon failure*). A confirmação desta falha é dada pelo *NT Status*, presente no *header* do SMB, que indica *STATUS_LOGON_FAILURE (0xc000006d)*, o que sugere que o nome de utilizador ou a *password* fornecida estão incorretos. De seguida, na *frame 659*, verifica-se uma resposta ACK padrão por parte do cliente, o que confirma o recebimento da mensagem do pacote anterior, que continha o erro de autenticação.

Uma vez falhada a autenticação, é utilizado o *comando Logoff AndX (0x74)* (na *frame 660*), enviado como parte do protocolo SMB, o que indica que o cliente está a encerrar a sessão com o servidor. A resposta (por parte do servidor) ao pedido de *logoff* feito pelo cliente no *frame* anterior encontra-se na *frame 661*. Esta apresenta um código de erro, *Bad userid*, o que sugere que o *logoff* não foi realizado com sucesso devido a algum problema relacionado com o utilizador. Como é possível observar na figura 33, o *User ID* tem como valor 0, o que significa que não há um utilizador autenticado ou que o servidor não reconheceu um *ID* válido para o processo de *logoff*, sendo, portanto, uma explicação para o erro gerado e anteriormente explicado.

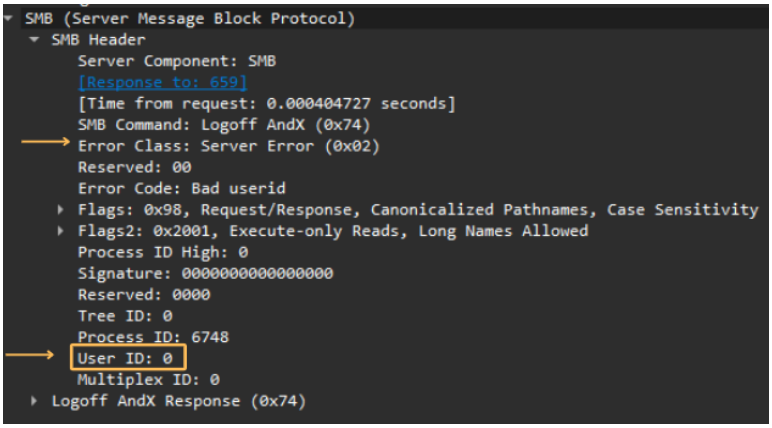


Figura 33 - Painel de detalhes do pacote da captura 3, da *frame* 661.

Por fim, o cliente dá início ao término da conexão, assinalado pela flag FIN (*frame 663*). Na resposta por parte do servidor (*frame* 664), na última *frame* deste fluxo TCP, é enviado um pacote RST (*reset*), o que significa que a conexão foi forçosamente encerrada, sem ocorrer o fechamento correto da comunicação como era esperado. Este comportamento pode ser justificado pela presença do erro de autenticação, e, desta forma, é possível garantir que a conexão seja fechada sem possibilidade de tentativas de comunicação após o erro. Dá-se assim por concluído o fluxo TCP.

Como dito anteriormente, este bloco de pacotes, referente ao fluxo de TCP explicado, repete-se desde a *frame 646* até à *frame 7552*, verificando-se sempre a ocorrência de um erro de autenticação e/ou *bad userId* juntamente com um término abrupto da conexão através do RST. Este comportamento sugere e reforça a ideia de que esta captura é referente a um ataque, e em específico para esta etapa de repetição de tentativa de login, corresponde a um ataque de força bruta com palavra-passe criptografada. Ou seja, o atacante pode estar a tentar múltiplas *passwords* criptografadas para tentar adivinhar a *password* correta para o utilizador John que a única credencial de acesso que se manteve constante ao longo de todas as tentativas de login.

Na figura 34, mais especificamente na *frame 7562*, verifica-se uma autenticação com sucesso por parte do cliente, o que sugere que o atacante descobriu a *password* associada ao utilizador John.

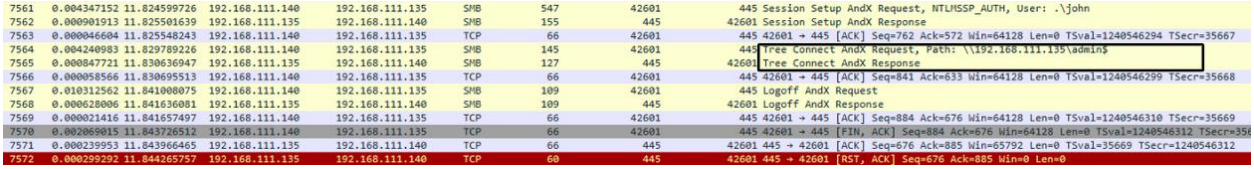


Figura 34 - Painel de visualização da captura 3, das *frames* 7561 – 7572.

Para além disso, na *frame 7564* surge a utilização de um novo comando SMB, o *Tree Connect AndX* (0x75), que é utilizado para estabelecer uma conexão com um recurso compartilhado.

O pacote da *frame 7564* faz parte da comunicação SMB entre dois sistemas na rede, onde o cliente está a tentar aceder o compartilhamento administrativo (admin$) no servidor, caminho: \\192.168.111.135\admin$. Foi utilizado o comando *Tree Connect AndX*, como já mencionado anteriormente, e, uma vez que possui um *status STATUS_SUCESS* é possível concluir que a solicitação foi bem-sucedida. Na *frame* seguinte (*frame* 7565), surge a resposta positiva à solicitação realizada.

Tendo acesso e conhecimento das credenciais corretas, o atacante é capaz de iniciar uma sessão SMB autenticada sem problemas. Surge então o processo de autenticação DCERPC, que tem como objetivo permitir que o cliente consiga executar serviços administrativos de forma remota. Este processo de autenticação DCERPC repete-se para cada uma das interfaces/serviços que o cliente pretende aceder, sendo estas (Figura 35):

- ***ISystemActivator;***
- ***IWBEMLevel1Login;***
- ***IRemUnknown;***
- ***IWBEMServices.***

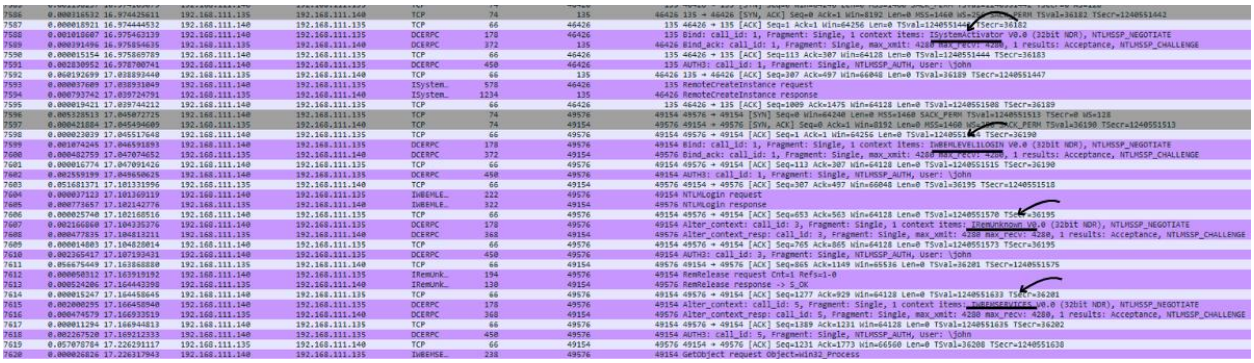


Figura 35 - Painel de visualização da captura 3, das *frames* 7587 - 7619. Foco nas interfaces onde se fez a autenticação.

Cada uma destas interfaces/serviços tem um propósito diferente, o *ISystemActivator* é utilizado para ativar componente COM (*Component Object Model*) de forma remota. Através deste serviço, o cliente pode solicitar a criação de objetos COM no servidor. Já a interface *WBEMLevel1Login* está associada ao WMI (*Windows Management Instrumentation*), que é uma infraestrutura de gestão de dados e operações em sistemas *Windows*. A autenticação nesta última interface é necessária para iniciar uma sessão e obter um *handler* (conexão) para aceder a outros serviços, como por exemplo o *IWBEMServices*, que permite que cliente realize consultas e operações de gestão. Por fim, a *IRemUnknown* é uma interface de utilizada no sentido de preparar um ambiente seguro para o acesso a outros serviços e interfaces.

A título de exemplo, a primeira autenticação inicia-se no pacote da *frame 7588*, com porta de origem: 46426 (escolhida aleatoriamente pelo sistema) e porta de destino: 135, que é a porta padrão para RPC (*Remote Procedure Call*) (protocolo a ser utilizado sobre o TCP). A comunicação envolve uma negociação de autenticação NTLMSSP (*NTLMSSP_NEGOTIATE*) com o serviço *ISystemActivator*. O tipo deste pacote é *Bind*, ou seja, é um processo de "ligar" o cliente ao servidor de modo que este possa realizar "*remote procedeures call"*. No *frame* seguinte (*frame 7589*), o servidor responde com "*Bind ack*" indicando, assim, a aceitação à solicitação. Dá-se então início ao *NTLMSSP_CHALLENGE*, analogamente ao que acontece no processo de autenticação SMB.

O modo/método de autenticação é semelhante para todas as interfaces, uma vez que o processo é baseado no NTLM, funcionando no sistema de desafio-resposta. Resumindo o processo descrito anteriormente para a autenticação ao *ISystemActivator*, as etapas básicas do *handshake* de autenticação são geralmente as seguintes:

1. ***Bind Request***: O cliente realiza uma solicitação de conexão com a interface/serviço que deseja, deixando explicito qual serviço quer aceder;
2. ***Challenge-Response***: O servidor responde com um desafio NTLM, e o cliente responde com as suas credenciais criptografadas (pelo menos a *password*);
3. **Aceitação da Autenticação**: Caso a resposta do cliente seja válida, o servidor concede o acesso.

Após realizar todas as autenticações necessárias, o cliente volta a estabelecer conexão com um recurso compartilhado, no mesmo caminho de *admin* que anteriormente tinha estabelecido. Uma

vez conectado, é utilizado um comando *NT Create AndX Request*, (*frame* 7650) o que sugere que houve uma solicitação para criar ou abrir um arquivo no compartilhamento *admin$*. No pacote do *frame 7652* encontra-se a resposta ao pedido relativo ao comando *NT Create AndX Request*, onde foi fornecido o acesso ao arquivo localizado em \\192.168.111.135\ADMIN$ com as permissões de leitura de atributos e de arquivos estendidos, porém, não existe indicação de acesso de escrita ou exclusão do arquivo.

Uma vez fornecido o acesso de abrir o arquivo, surge um novo comando *Trans2* que permite realizar consulta ou modificações adicionais no arquivo aberto. Juntamente com este comando, está o comando *QUERY_FILE_INFO* que serve para obter informações sobre o arquivo. Neste contexto, o comando *Trans2: QUERY_FILE_INDO* foi utilizado para recuperar informações padrão (metadados) de um arquivo específico no servidor SMB, identificado pelo FID 0x4001 (o arquivo de nome temporário _1647810071.8674524). Após realizar a consulta, surge um *Close Request* seguido de um *Tree Disconnect Request*, terminando assim a conexão com o recurso partilhado. Este é basicamente a estrutura de como se processa a conexão com um recurso partilhado, podendo existir ainda, dentro da conexão, alguns comandos de interesse como por exemplo:

- ***Read AndX Request*;**

Na *frame 7736* é realizado um pedido para ler a informação do conteúdo do arquivo, para tal é utilizado o comando *Read AndX Request*, estando a resposta para este pedido presente no pacote do *frame 7737*. Escolheu-se este *frame* como exemplo pois é o único nesta captura que apresenta um conteúdo legível, e para fins ilustrativos torna-se um bom exemplo, podendo ser visto na figura 36.

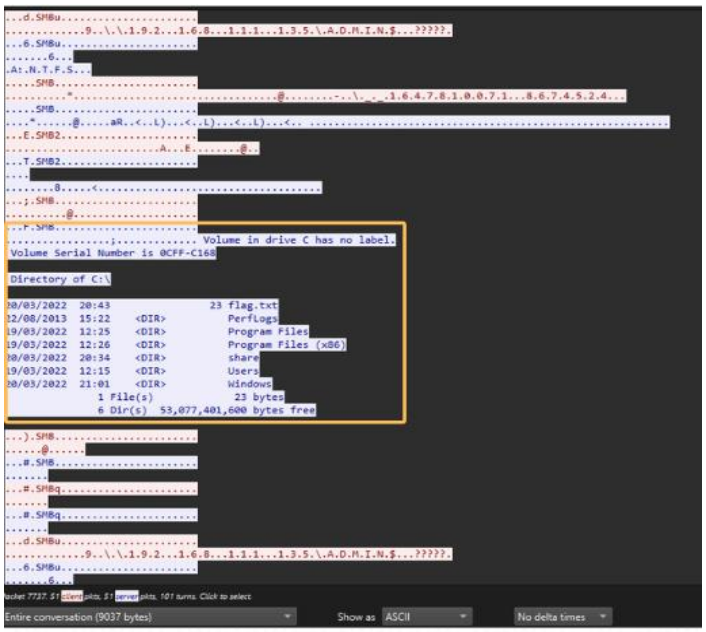


Figura 36 - TCP *stream* da captura 3.

- **Trans2: FIND_FIRST2;**

Os pacotes que utilizam o comando *SMB Trans2* com o subcomando *FIND_FIRST2*, procuram arquivos específicos com atributos como somente leitura, ocultos e de sistema, e utilizam um padrão de pesquisa. Exemplo da utilização deste comando encontra-se no pacote do *frame 7783*, onde o padrão de pesquisa apresentava um conjunto de caracteres não *ASCII*, como é possível verificar na figura 37.

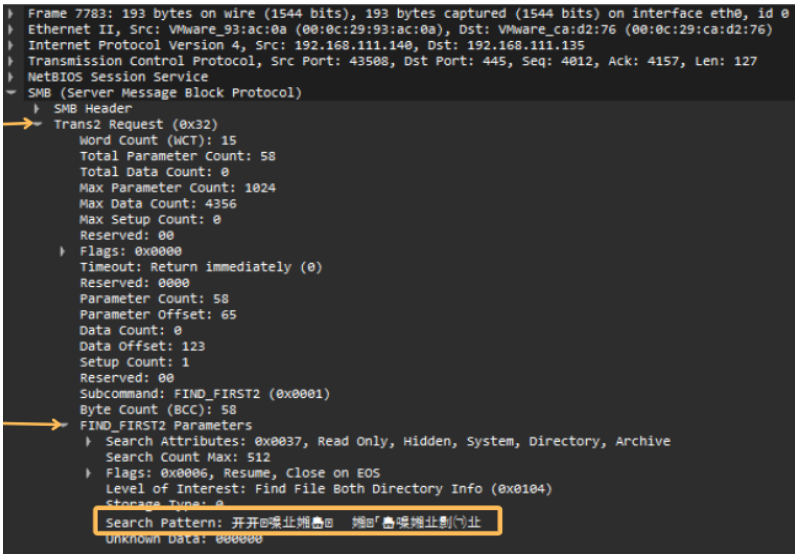


Figura 37 - Painel de detalhes do pacote da captura 3, da *frame* 7783.

- **Write AndX Response.**

Este comando é comummente utilizado para a manipulação de arquivos, como a gravação de dados num arquivo de forma remota. No caso de exemplo do pacote do *frame 7834* (Figura 38) verifica-se que foram gravados 76 bytes de informação num arquivo **\srvsvc.**

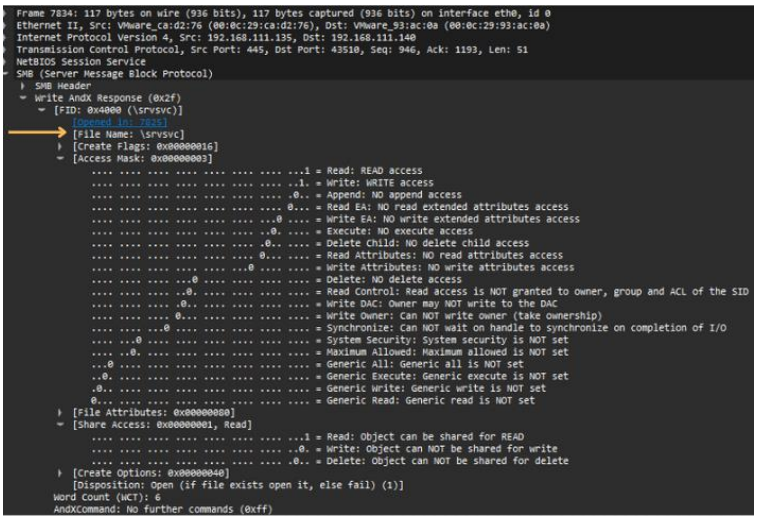


Figura 38 - Painel de detalhes do pacote da captura 3 do *frame* 7834.

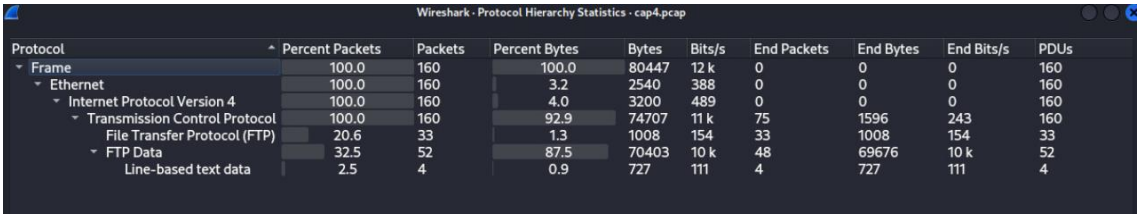
Dando continuidade e conclusão à ideia de esta captura se trata de um ataque, esta etapa final explicada pelos diversos comandos de consulta e manipulação de dados, confirma a sugestão inicial de que se trata de um ataque. Poder-se inferir que este ataque seja um ataque de exploração ou coleta de informação dentro da rede uma vez que o atacante não só explorou/mapeou a rede (primeira fase), como forçou a autenticação com várias tentativas falhadas até conseguir penetrar na rede (segunda fase). Após explorar as vulnerabilidades no protocolo SMB, tentou obter informações sobre arquivos presentes na rede (terceira fase).

2.4 Captura 4

2.4.1 Protocolo Principal

Ao analisar a captura 4 percebe-se que os protocolos principais são o FTP e TCP.

Clicando em *Statistics* e de seguida, em *Protocol Hierarchy*, é possível observar que os protocolos presentes na captura são os protocolos TCP e FTP, como demonstrado na figura 39.



Wireshark · Protocol Hierarchy Statistics · cap4.pcap

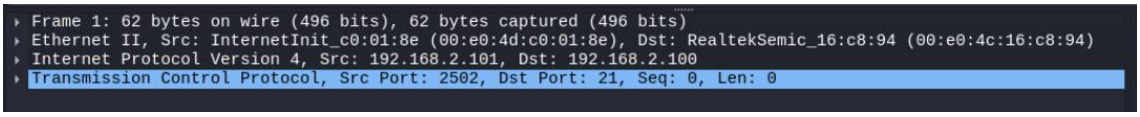
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	160	100.0	80447	12 k	0	0	0	160
Ethernet	100.0	160	3.2	2540	388	0	0	0	160
Internet Protocol Version 4	100.0	160	4.0	3200	489	0	0	0	160
Transmission Control Protocol	100.0	160	92.9	74707	11 k	75	1596	243	160
File Transfer Protocol (FTP)	20.6	33	1.3	1008	154	33	1008	154	33
FTP Data	32.5	52	87.5	70403	10 k	48	69676	10 k	52
Line-based text data	2.5	4	0.9	727	111	4	727	111	4

Figura 39 - Visualização de estatísticas da hierarquia de protocolos da captura 4, no *Wireshark*.

2.4.2 Análise das Máquinas envolvidas na Captura

As máquinas envolvidas são as máquinas correspondes ao cliente e ao servidor.

Nas informações correspondentes à captura, a secção *Ethernet II* mostra os endereços MAC da fonte e destino para a captura analisada. Os endereços IP e as portas do cliente e do servidor, no caso da *frame* 1, onde se inicia a conexão TCP, encontram-se nas secções *Internet Protocol Version 4* e *Transmission Control Protocol*, respetivamente (Figura 40).



```
▸ Frame 1: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
▸ Ethernet II, Src: InternetInit_c0:01:8e (00:e0:4d:c0:01:8e), Dst: RealtekSemic_16:c8:94 (00:e0:4c:16:c8:94)
▸ Internet Protocol Version 4, Src: 192.168.2.101, Dst: 192.168.2.100
▸ Transmission Control Protocol, Src Port: 2502, Dst Port: 21, Seq: 0, Len: 0
```

Figura 40 – Painel de detalhes do pacote da captura 4, da *frame* 1.

Para facilitar a visualização destes dados, os mesmos, estão registados na Tabela 4.

Tabela 4 - Informações referentes aos MAC *address* e IP *adress* de ambas as máquinas envolvidas na captura.

	MAC *Address*	**IP *Address***
Cliente	00:e0:4d:c0:01:8e, associado a um dispositivo identificado como InternetInit_c0:01:8e	192.168.2.101
Servidor	00:e0:4c:16:c8:94, associado a um dispositivo identificado como RealtekSemic_16:c8:94	192.168.2.100

2.4.3 Comandos e Funcionalidades solicitadas

O cliente inicia uma conexão TCP da porta de origem 2502 (*source port*) com o servidor na porta 21 (porta padrão para FTP) enviando uma flag SYN (*frame* 1). O servidor responde com SYN e ACK (*frame* 2), e o cliente confirma com ACK (*frame* 3), completando o *Three-Way Handshake*, explicado na Captura 1, (Figura 41). Assim, é estabelecida uma conexão confiável para a troca de dados entre cliente e servidor.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.2.101	192.168.2.100	TCP	62	2502 → 21 [SYN] Seq=0 Win=16384 Len=0 MSS=1460 SACK_
2	0.000000	192.168.2.100	192.168.2.101	TCP	62	21 → 2502 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS
3	0.000000	192.168.2.101	192.168.2.100	TCP	60	2502 → 21 [ACK] Seq=1 Ack=1 Win=17520 Len=0
4	0.001000	192.168.2.100	192.168.2.101	FTP	112	Response: 220 colasoft_test_2 Microsoft FTP Service
5	0.158000	192.168.2.101	192.168.2.100	TCP	60	2502 → 21 [ACK] Seq=1 Ack=59 Win=17462 Len=0
6	3.996000	192.168.2.101	192.168.2.100	FTP	74	Request: USER administrator
7	3.997000	192.168.2.100	192.168.2.101	FTP	96	Response: 331 Password required for administrator.
8	4.163000	192.168.2.101	192.168.2.100	TCP	60	2502 → 21 [ACK] Seq=21 Ack=101 Win=17420 Len=0
9	6.108000	192.168.2.101	192.168.2.100	FTP	67	Request: PASS 123456
10	6.111000	192.168.2.100	192.168.2.101	FTP	97	Response: 230-Welcome to colasoft_test site,thanks!
11	6.266000	192.168.2.101	192.168.2.100	TCP	60	2502 → 21 [ACK] Seq=34 Ack=144 Win=17377 Len=0
12	6.266000	192.168.2.100	192.168.2.101	FTP	89	Response: 230 User administrator logged in.

Figura 41 – Painel de visualização da captura 4, das *frames* 1-12.

Na *frame* 4, o servidor envia ao cliente uma mensagem "220" de boas-vindas (código de resposta FTP), indicando que está ativo e pronto para conexão. De seguida, o cliente confirma que recebeu com um pacote TCP com a flag ACK (*frame* 5) e envia o comando USER com o nome "*administrator*" (*frame* 6), iniciando a autenticação. Após isso, envia o comando PASS com a senha "123456", pedindo ao servidor que aceite as credenciais. O servidor valida as credenciais e responde com mensagens de sucesso (*frames* 10 e 12), autenticando o utilizador [26].

O cliente então usa o comando PORT para abrir a porta 2504 (o comando PORT é dado por PORT h1,h2,h3,h4,p1,p2, através do qual o número da porta é dado por p1×256+p2 = 9×256+200 = 2504) [27] para a transferência de dados (*frames* 14 e 15) e solicita ao servidor uma lista de arquivos com o comando NLST (*frame* 16), (Figura 42) [26]. O servidor confirma a conexão para a transferência (*frame* 17) e, como as portas utilizadas são 2502 e 21, é necessário um novo *Three-Way Handshake* (*frames* 18-20, Figura 42).

No.	Time	Source	Destination	Protocol	Length	Info
14	7.772000	192.168.2.101	192.168.2.100	FTP	80	Request: PORT 192,168,2,101,9,200
15	7.780000	192.168.2.100	192.168.2.101	FTP	84	Response: 200 PORT command successful.
16	7.784000	192.168.2.101	192.168.2.100	FTP	60	Request: NLST
17	7.784000	192.168.2.100	192.168.2.101	FTP	109	Response: 150 Opening ASCII mode data connection for file
18	7.784000	192.168.2.100	192.168.2.101	TCP	62	20 → 2504 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM
19	7.784000	192.168.2.101	192.168.2.100	TCP	62	2504 → 20 [SYN, ACK] Seq=0 Ack=1 Win=17520 Len=0 MSS=1460
20	7.784000	192.168.2.100	192.168.2.101	TCP	54	20 → 2504 [ACK] Seq=1 Ack=1 Win=65535 Len=0
21	7.786000	192.168.2.100	192.168.2.101	FTP-DA…	132	FTP Data: 78 bytes (PORT) (NLST)
22	7.792000	192.168.2.100	192.168.2.101	TCP	54	20 → 2504 [FIN, ACK] Seq=79 Ack=1 Win=65535 Len=0
23	7.799000	192.168.2.101	192.168.2.100	TCP	60	2504 → 20 [ACK] Seq=1 Ack=80 Win=17442 Len=0
24	7.803000	192.168.2.101	192.168.2.100	TCP	60	2504 → 20 [FIN, ACK] Seq=1 Ack=80 Win=17442 Len=0
25	7.803000	192.168.2.100	192.168.2.101	TCP	54	20 → 2504 [ACK] Seq=80 Ack=2 Win=65535 Len=0
26	7.969000	192.168.2.101	192.168.2.100	TCP	60	2502 → 21 [ACK] Seq=66 Ack=264 Win=17257 Len=0
27	7.969000	192.168.2.100	192.168.2.101	FTP	78	Response: 226 Transfer complete.

Figura 42 – Painel de visualização da captura 4, das *frames* 14-27.

Na *frame* 21, o servidor envia a lista de arquivos do diretório solicitada pelo cliente (Figura 43), e a porta usada para transferência de dados pelo protocolo FTP-DATA é a porta 20 [26].

```
▸ Frame 21: 132 bytes on wire (1056 bits), 132 bytes captured (1056 bits)
▸ Ethernet II, Src: RealtekSemic_16:c8:94 (00:e0:4c:16:c8:94), Dst: InternetInit_c0:01:8e (00:e0:4d:c0:01
▸ Internet Protocol Version 4, Src: 192.168.2.100, Dst: 192.168.2.101
▸ Transmission Control Protocol, Src Port: 20, Dst Port: 2504, Seq: 1, Ack: 1, Len: 78
  FTP Data (78 bytes data)
  [Setup frame: 14]
  [Setup method: PORT]
  [Command: NLST]
  Command frame: 16
  [Current working directory: ]
▾ Line-based text data (6 lines)
    laprxy.dll\r\n
    logagent.exe\r\n
    mplayer2.exe\r\n
    npds.zip\r\n
    npdsplay.dll\r\n
    npwmsdrm.dll\r\n
```

Figura 43 – Painel de detalhes do pacote da captura 4, da *frame* 21.

Na *frame* 27, o servidor envia o código 226, confirmando que a transferência de dados foi concluída com sucesso e a conexão de dados encerrada. O cliente então solicita a abertura da porta 2505 (*frame* 29), que o servidor confirma com o código 200 (*frame* 30), permitindo nova transferência de dados. Em seguida, o cliente usa o comando LIST (*frame* 31) para solicitar uma listagem detalhada dos arquivos e diretórios, incluindo permissões e tamanhos, ao contrário do comando NLST, que retorna apenas os nomes [26]. O servidor responde com o código 150 (*frame* 32), sinalizando a abertura da conexão de dados. Após um novo *Three-Way Handshake* (*frames* 33-35), o cliente recebe a listagem detalhada dos arquivos (*frame* 36), Figura 44.

```
▾ Line-based text data (6 lines)
    06-19-03  12:05PM                26896 laprxy.dll\r\n
    06-19-03  12:05PM                65296 logagent.exe\r\n
    06-19-03  12:05PM                 4639 mplayer2.exe\r\n
    01-10-00  08:00PM                22060 npds.zip\r\n
    06-19-03  12:05PM               364544 npdsplay.dll\r\n
    01-10-00  08:00PM                 8464 npwmsdrm.dll\r\n
```

Figura 44 – Lista recebida, na *frame* 36.

Depois, a conexão da porta 2505 termina com o método FIN e ACK do protocolo TCP (Figura 45) [5]. O servidor informa que a transferência de dados está completa com o código 226 (*frame* 42).

29	9.447000	192.168.2.101	192.168.2.100	FTP	80	Request: PORT 192,168,2,101,9,201
30	9.447000	192.168.2.100	192.168.2.101	FTP	84	Response: 200 PORT command successful.
31	9.451000	192.168.2.101	192.168.2.100	FTP	60	Request: LIST
32	9.451000	192.168.2.100	192.168.2.101	FTP	107	Response: 150 Opening ASCII mode data connection for /bi
33	9.451000	192.168.2.100	192.168.2.101	TCP	62	20 → 2505 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM
34	9.452000	192.168.2.101	192.168.2.100	TCP	62	2505 → 20 [SYN, ACK] Seq=0 Ack=1 Win=17520 Len=0 MSS=146
35	9.452000	192.168.2.100	192.168.2.101	TCP	54	20 → 2505 [ACK] Seq=1 Ack=1 Win=65535 Len=0
36	9.452000	192.168.2.100	192.168.2.101	FTP-DA…	366	FTP Data: 312 bytes (PORT) (LIST)
37	9.452000	192.168.2.100	192.168.2.101	TCP	54	20 → 2505 [FIN, ACK] Seq=313 Ack=1 Win=65535 Len=0
38	9.453000	192.168.2.101	192.168.2.100	TCP	60	2505 → 20 [ACK] Seq=1 Ack=314 Win=17208 Len=0
39	9.470000	192.168.2.101	192.168.2.100	TCP	60	2505 → 20 [FIN, ACK] Seq=1 Ack=314 Win=17208 Len=0
40	9.470000	192.168.2.100	192.168.2.101	TCP	54	20 → 2505 [ACK] Seq=314 Ack=2 Win=65535 Len=0
41	9.571000	192.168.2.101	192.168.2.100	TCP	60	2502 → 21 [ACK] Seq=98 Ack=371 Win=17150 Len=0
42	9.571000	192.168.2.100	192.168.2.101	FTP	78	Response: 226 Transfer complete.

Figura 45 – Painel de visualização da captura 4, das *frames* 29-42.

É novamente aberta uma porta, desta vez a 2506 (*frame* 44) e recebida a resposta de sucesso do pedido (*frame* 45 com o código 200).

Na *frame* 46, o cliente envia o comando RETR, com o objetivo de descarregar uma cópia da aplicação *logagent.exe*, com o peso de 65296 *bytes* (Figura 46) [26]. Na *frame* seguinte, aparece a resposta do servidor com o código 150, mostrando que a conexão pode ser feita.

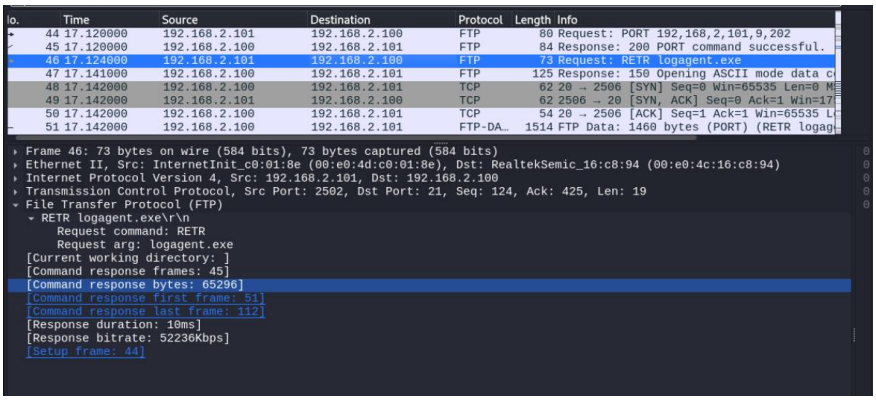


Figura 46 – Painel de visualização da captura 4, *frames* 44-51, e, painel de detalhes da *frame* 46.

Após a conexão TCP entre as portas 20 e 2506 (*frames* 48-50), o servidor envia ao cliente 44 pacotes FTP-DATA de 1460 bytes cada (*frames* 51-112) e um último de 1056 bytes (*frame* 112), totalizando 65296 bytes, que correspondem ao arquivo *logagent.exe*. Durante a transferência, o cliente envia confirmações ACK (como nas *frames* 98, 101 e 104). As linhas escuras indicam que o *buffer* de receção para a transferência do ficheiro se encontra saturado (Figura 47).

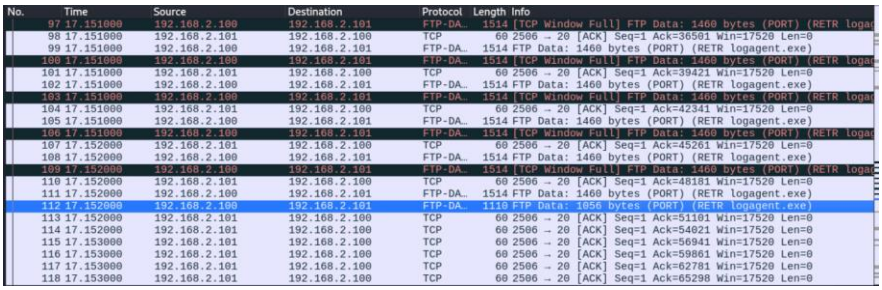


Figura 47 – Painel de visualização da captura 4, das *frames* 97-118.

Na *frame* 119, a conexão entre as portas 2506 e 20 é finalizada com um FIN ACK, e o cliente e o servidor retornam às portas 21 e 2502. O servidor envia o código 226 (*frame* 122), confirmando o fim da transferência. De seguida, o cliente solicita a abertura da porta 2507 (*frame* 124) para transferir o arquivo *mplayer2.exe* (4639 bytes), na figura 48. Como o TCP-DATA suporta 1460 bytes por pacote, são enviados três pacotes completos (*frames* 131, 132 e 134) e um último de 259 bytes (*frame* 135). Após a transferência, as portas iniciais são restauradas com um novo FIN ACK (*frame* 138).

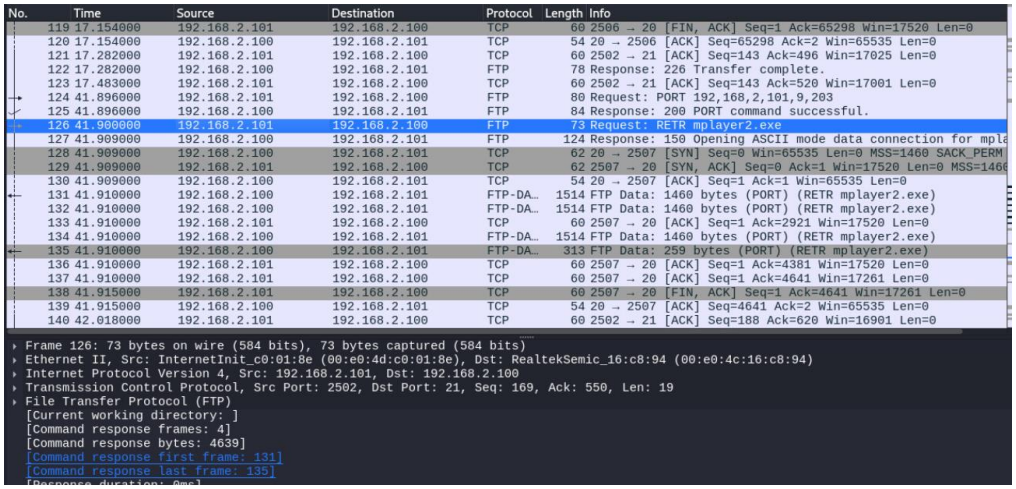


Figura 48 – Painel de visualização da captura 4, *frames* 119-140, e, painel de detalhes da *frame* 126.

Na *frame* 143, é enviado, pelo cliente, o comando XPWD que solicita o diretório atual [28], ao qual recebe a resposta do servidor com o código 257, indicando que se encontra no diretório atual "/" (Figura 49, *frame* 144). O cliente volta a abrir uma nova porta (2508) e envia o comando NLST (*frame* 148), após confirmação do sucesso pelo servidor. Através do FTP-DATA, o servidor envia para o cliente a lista de nomes dos ficheiros que se encontram no diretório atual (*frame* 153, Figura 49).

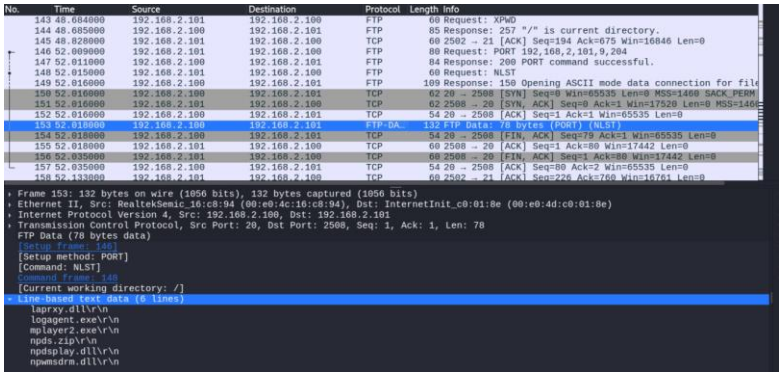


Figura 49 – Painel de visualização da captura 4, *frames* 143-158, e, painel de detalhes da *frame* 153.

A transferência de dados é terminada com sucesso após o regresso às portas iniciais, pelo método FIN e ACK (*frame* 158).

Resumindo de forma mais eficaz a comunicação TCP que ocorre nesta captura, podemos obter o resumo através da *TCP stream*, do *wireshark*, sendo que as atividades referentes ao servidor estão representadas a azul, e ao cliente a vermelho (Figura 50).

```
220 colasoft_test_2 Microsoft FTP Service (Version 5.0).
USER administrator
331 Password required for administrator.
PASS 123456
230-Welcome to colasoft_test site,thanks!
230 User administrator logged in.
PORT 192,168,2,101,9,200
200 PORT command successful.
NLST
150 Opening ASCII mode data connection for file list.
226 Transfer complete.
PORT 192,168,2,101,9,201
200 PORT command successful.
LIST
150 Opening ASCII mode data connection for /bin/ls.
226 Transfer complete.
PORT 192,168,2,101,9,202
200 PORT command successful.
RETR logagent.exe
150 Opening ASCII mode data connection for logagent.exe(65296 bytes).
226 Transfer complete.
PORT 192,168,2,101,9,203
200 PORT command successful.
RETR mplayer2.exe
150 Opening ASCII mode data connection for mplayer2.exe(4639 bytes).
226 Transfer complete.
XPWD
257 "/" is current directory.
PORT 192,168,2,101,9,204
200 PORT command successful.
NLST
150 Opening ASCII mode data connection for file list.
226 Transfer complete.
```

Figura 50 - TCP *stream* da captura 4.

2.5 Captura 5

Analisando a captura 5 é possível afirmar que o comportamento observado sugere uma tentativa de exploração do servidor *BadBlue*. Isto é, primeiramente, é estabelecida uma conexão TCP entre as máquinas do atacante (IP 192.168.155.226 e endereço MAC 00:0c:29:76:16:1a) e da vítima (IP 192.168.155.232 e endereço MAC 00:0c:29:f2:b6:0b) na porta 80, estabelecendo o *Three-Way Handshake* (*frames* 1-5).

De seguida, o atacante realiza uma requisição HTTP GET para um arquivo suspeito (*ext.dll*) [26] com o comando *PassThru*, possivelmente tentando explorar uma vulnerabilidade de execução remota de comandos (RCE) no servidor [29]. Isto é concluído porque o *User-Agent* da solicitação HTTP está configurado como *Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)*, que representa o *Internet Explorer 6* num sistema *Windows* XP. O uso deste *User-Agent* é incomum em ambientes modernos, onde a maioria dos acessos legítimos ocorre a partir de navegadores mais atuais [30]. Assim, o seu uso sugere que o atacante pode estar a tentar disfarçar a conexão, como proveniente de um sistema desatualizado e obsoleto, possivelmente para evitar deteção por medidas de segurança. Esta é uma técnica conhecida como *spoofing*, nomeadamente, *User-agent spoofing*, que ajuda a disfarçar a atividade maliciosa e a evitar a ativação de regras de segurança configuradas para detetar tráfego suspeito, ou seja, a intenção do mesmo é responder a uma solicitação de forma maliciosa [31].

Desta forma, a requisição HTTP GET com parâmetros maliciosos, a par do uso de um *User-Agent* desatualizado, reforça a hipótese de que a atividade observada pode ser maliciosa e de que a conexão estabelecida teve o intuito de explorar vulnerabilidades no servidor.

Após a tentativa de exploração, ocorre um tráfego intenso de pacotes TCP entre os IPs, com *flags* PSH e ACK (a partir da *frame* 13), sugerindo que o atacante pode estar a transferir dados ou a utilizar a conexão para explorar informações/monitorizar o servidor. Este padrão de tráfego intenso é compatível com uma exploração bem-sucedida, permitindo ao atacante interagir com o sistema.

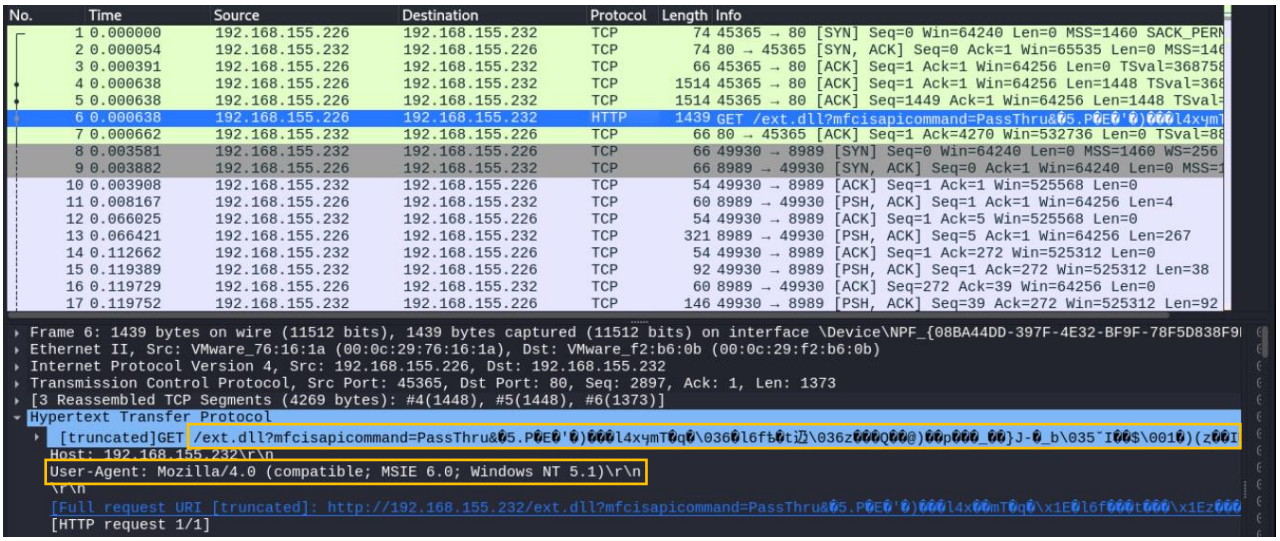


Figura 51 – Painel de visualização da captura 5, *frames* 1-17, e, painel de detalhes da *frame* 6.

A partir da *frame* 24, é executado o comando <dir> dentro do diretório C:*\Program Files (x86)\BadBlue\EE*, listando o conteúdo da pasta, ao longo das *frames* seguintes. Entre os arquivos listados, existem alguns suspeitos e comuns em ferramentas de exploração como *badblue.exe* (principal executável) e *BadBlue Enterprise Edition.url* (atalho para a documentação de uma versão do BadBlue) – ambos na *frame* 63 (Figura 52). Para além disso são enviados outros arquivos .bat, que são scripts que podem ser usados para interagir com o servidor e, ainda, aquivos .hts, .htx, .gif, .css, .exe, que podem ser componente do servidor web, como páginas HTML, scripts ou executáveis de suporte.

Verificando a data dos arquivos (2003, 2004, 2006) podemos dizer que os mesmos não foram modificados recentemente, podendo significar que o *software BadBlue* foi instalado há muito tempo e possivelmente não foi atualizado, o que poderia deixá-lo vulnerável a ataques conhecidos.

Desta forma, podemos dizer que como o *BadBlue* não recebe atualizações de segurança, está suscetível a várias vulnerabilidades, incluindo falhas de execução remota como a descrita com *ext.dll* e *PassThru*.

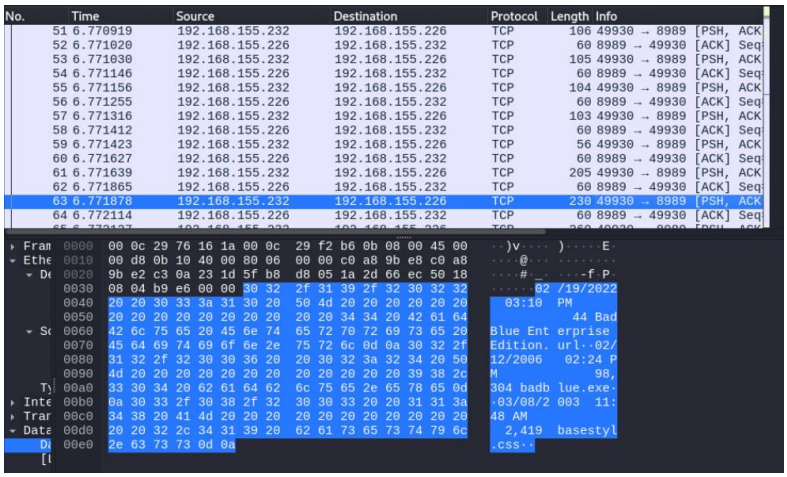


Figura 52 - Painel de visualização da captura 5, *frames* 51-64, e, painel de detalhes da *frame* 63.

Posteriormente, nas *frames* 154-155, a captura de pacotes mostra pacotes ICMPv6 de *Router Solicitation e Router Advertisement*. Isso indica que o dispositivo 192.168.155.232 está a solicitar informações de configuração IPv6 a partir de um router disponível na rede e que, um router ou outro dispositivo respondeu à solicitação, informando sobre as configurações e parâmetros de rede IPv6 como o endereço da *gateway* padrão, por exemplo. Relacionando esta informação com o contexto anterior de ataque, podemos concluir que esta comunicação estaria a ser manipulada pelo atacante, controlando o router ou interferindo nas respostas.

Após isso, a vítima (192.168.155.232) tenta comunicar-se com o dispositivo com IP 192.168.155.101 através de uma solicitação ARP, para obter o seu endereço MAC e estabelecer uma conexão. Por fim, o dispositivo com IP 192.168.155.101 responde à solicitação, revelando qual o seu endereço MAC.

Caso o dispositivo que fizesse o pedido ARP com o IP da vítima (192.168.155.232) mas enviando o endereço MAC do atacante, poderia tratar-se de um início de ataque *man-in-the-middle*, através de *ARP Spoofing*. O ARP Spoofing é um tipo de ataque que faz pedidos ARP falsos ou respostas ARP falsas. Como demonstra a figura 53, esta atividade poderia tratar-se de ARP Request Spoofing se o atacante com o endereço MAC C:C:C:C:C enviasse um pedido ARP falso, incluindo uma resposta falsa indicando que tem o endereço MAC F:F:F:F:F, convencendo a vítima

de que o IP 10.0.0.1 estaria associado ao seu IP, que é o controlado pelo atacante [32]. Assim, conseguiria executar um ataque *man-in-the-middle*, conseguindo intercetar, alterar, substituir ou modificar os dados transmitidos no canal de comunicação entre os pontos finais. As vítimas acreditam que o canal de comunicação está protegido, pois desconhecem a presença destes intrusos [33].

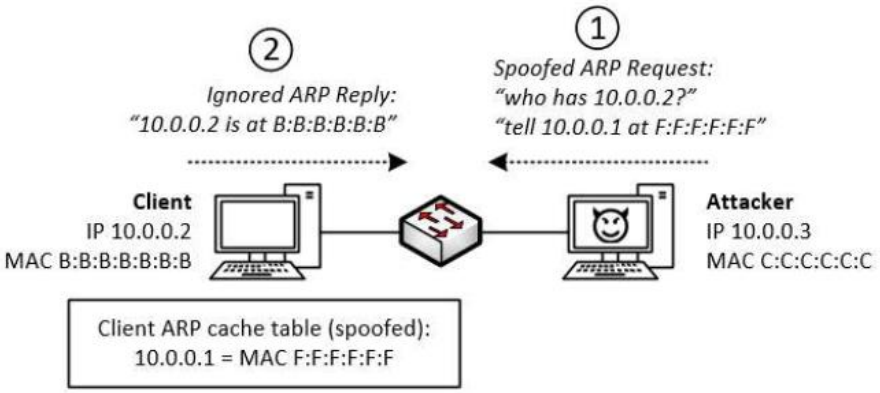


Figura 53 - *ARP Request Spoofing* [32].

Contudo, na captura 5 verificamos que o pedido ARP na *frame* 156 é feito pelo IP e endereço MAC da vítima (00:0c:29:f2:b6:0b) e que, na *frame* 157, o dispositivo que responde envia o seu endereço MAC para o endereço MAC da vítima, como se vê na figura 54.

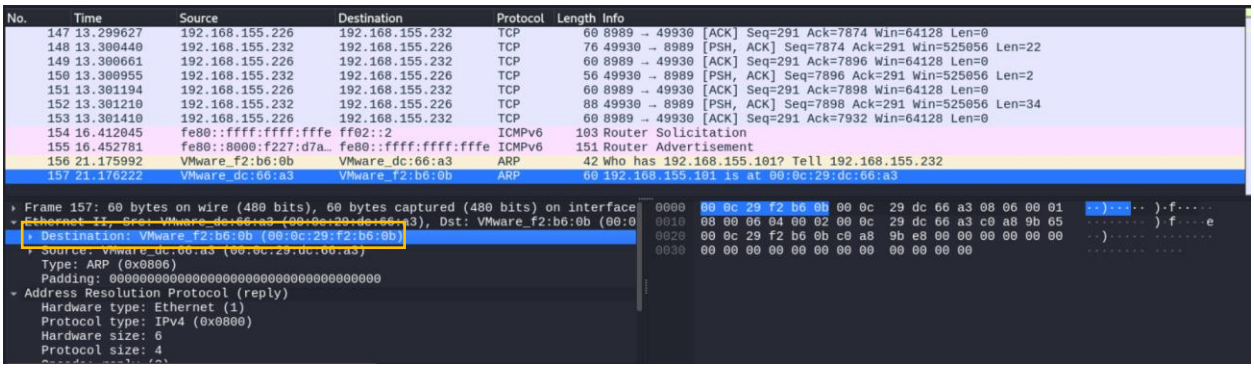


Figura 54 - Painel de visualização da captura 5, *frames* 147-157, e, painel de detalhes da frame 157.

2.6 Captura 6

No âmbito da análise da captura 6 é estabelecida uma conexão TCP com IP 192.168.155.226 (cliente) e MAC Address 00:0c:29:76:16:1a na porta 45449, estabelecendo o *Three-Way Handshake*, com o servidor com IP 192.168.155.232 e MAC Address 00:0c:29:f2:b6:0b na porta 445 (*frames* 1-3).

Primeiramente, o cliente e o servidor negociam a versão do protocolo SMB a ser utilizada para a sessão (*frames* 4-9). De seguida, o cliente envia um Session Setup Request (*frame* 10) com autenticação NTLMSSP (NT LAN *Manager Security Support Provider*) para iniciar a sessão. Em resposta, o servidor retorna com um STATUS_MORE_PROCESSING_REQUIRED (*frame* 11), o que indica que é necessário mais processamento antes que a autenticação seja concluída.

Posteriormente, o cliente estabelece uma sessão SMB2 com o servidor, autenticando o utilizador ".\mcalister" (*frame* 13). De seguida, nas *frames* 16-21 ocorreram tentativas de acesso a recursos partilhados no servidor. Na *frame* 16, ocorre um *Tree Connect Request* (o cliente solicita o acesso a uma partilha específica no servidor), enviando o IPC$ (*Inter-Process Communication*). Este trata-se de um recurso utilizado principalmente para facilitar a comunicação e a partilha de dados entre aplicações no sistema, como envio de comandos e obtenção de informações do sistema remoto [34]. Nas *frames* 19 e 20, por sua vez, o pedido "*Tree Connect Request*" para o recurso "\empty" significa que o cliente obteve permissão para aceder a essa pasta, que se encontra no sistema de ficheiros do servidor.

Seguidamente, o cliente tenta aceder ou criar um arquivo powershell.exe (*frame* 22), porém, o servidor indica que o caminho ou o arquivo não existe no mesmo (*frame* 23). Após a tentativa mal-sucedida de aceder ao arquivo, o cliente desconecta, utilizando o comando "Tree Disconnect Request" e encerra a sessão com o servidor (*frame* 25-26). O pacote de solicitação "SMB2 TREE_DISCONNECT" é enviado pelo cliente para solicitar que a conexão da árvore especificada no TreeId no cabeçalho SMB2 seja desconectada [35].

De seguida, o cliente cria um arquivo chamado "tGywfrOy.exe" no servidor. A operação "Create Request" – *frame* 31 – indica uma tentativa de criação de arquivo, e a "Create Response" – *frame* 32 – confirma que o arquivo foi criado com sucesso. Após a criação, verificam-se várias operações de escrita no ficheiro criado (*frames* 33-99). Cada pedido de escrita tem um

comprimento específico (*lenght* = 500 bytes) e um deslocamento (Off), que indica a posição do byte no arquivo onde o bloco de dados é gravado. O deslocamento aumenta 500 bytes a cada vez, mostrando gravações sequenciais no arquivo. Nas *frames* 100-103, o cliente fecha o ficheiro "tGywfrOy.exe" e desconecta.

Posteriormente, ocorre novamente um pedido de conexão a um recurso específico em "\\192.168.155.232\IPC$", o qual foi aceite pelo servidor (*frames* 104-105). O cliente realiza um pedido de criação de um ficheiro "svcctl", novamente aceite pelo servidor (*frames* 106-107). De seguida, são escritos dados no ficheiro partilhado.

Imediatamente, são abordados os protocolos DCE/RPC (*Distributed Computing Environment / Remote Procedure Calls*) e SVCCTL (*Service Control Manager Remote Protocol*) (*frames* 110-126). Como se pode observar no excerto presente na figura 55, o cliente tenta manipular serviços no servidor, através de operações SVCCTL. O fluxo inclui a conexão ao SCM ("OpenSCManagerW"), a criação de um serviço ("*CreateServiceW*") e o seu início ("StartServiceW"). As operações SMB2 indicam leitura e escrita no ficheiro "svcctl", necessárias para o controlo remoto dos serviços.

106	0.205732	192.168.155.226	192.168.155.232	SMB2	206	Create Request File: svcctl
107	0.205857	192.168.155.232	192.168.155.226	SMB2	222	Create Response File: svcctl
108	0.210254	192.168.155.226	192.168.155.232	SMB2	846	Write Request Len:664 Off:0 File: svcctl
109	0.210335	192.168.155.232	192.168.155.226	SMB2	150	Write Response
110	0.212891	192.168.155.226	192.168.155.232	DCERPC	206	Bind: call_id: 0, Fragment: Single, 15 context items: b8c27029-437f-f93f-cb
111	0.213048	192.168.155.232	192.168.155.226	SMB2	150	Write Response
112	0.215472	192.168.155.226	192.168.155.232	SMB2	183	Read Request Len:65535 Off:0 File: svcctl
113	0.215573	192.168.155.232	192.168.155.226	DCERPC	554	Bind_ack: call_id: 0, Fragment: Single, max_xmit: 4280 max_recv: 4280, 15 r
114	0.218198	192.168.155.226	192.168.155.232	SVCCTL	266	OpenSCManagerW request
115	0.218277	192.168.155.232	192.168.155.226	SMB2	150	Write Response
116	0.220763	192.168.155.226	192.168.155.232	SMB2	183	Read Request Len:65535 Off:0 File: svcctl
117	0.220814	192.168.155.232	192.168.155.226	SVCCTL	198	OpenSCManagerW response
118	0.224042	192.168.155.226	192.168.155.232	SVCCTL	430	CreateServiceW request
119	0.224130	192.168.155.232	192.168.155.226	SMB2	150	Write Response
120	0.226482	192.168.155.226	192.168.155.232	SMB2	183	Read Request Len:65535 Off:0 File: svcctl
121	0.226536	192.168.155.232	192.168.155.226	SVCCTL	202	CreateServiceW response
122	0.229007	192.168.155.226	192.168.155.232	SVCCTL	234	StartServiceW request
123	0.229080	192.168.155.232	192.168.155.226	SMB2	150	Write Response

Figura 55 - Aplicação de operações SVCCTL.

Observa-se, posteriormente, um TCP *Three-Way Handshake* (*frames* 127-129), sendo que a máquina cujo IP é 192.168.155.232, na porta 49752 estabelece uma conexão com a máquina 192.168.155.226 na porta 4444. Assim, a comunicação muda para a porta 4444, que pode ser utilizada para estabelecer um novo serviço ou canal de comunicação. Tal pode ter sido utilizado para separar o tráfego relacionado ao novo serviço do tráfego SMB inicial.

De seguida, o fluxo de pacotes PSH/ACK sugere que comandos ou informações estão a ser transmitidos e processados de imediato (*frames* 129-146). No final da sequência, observa-se um pedido "DeleteService" (*frame* 147), realizado pelo cliente. A sequência de *frames* sugere que o

cliente está a tentar remover um serviço que ele próprio criou anteriormente. O uso do comando "DeleteService" após "CreateServiceW" e "StartServiceW" indica que o cliente pretende apagar qualquer vestígio da manipulação que foi feita. Tal pode indicar uma técnica de *malware fileles*, onde um atacante cria serviços temporários para executar comandos e depois remove o serviço para ocultar a atividade. O *malware fileless* é escrito diretamente na memória em vez de ser armazenado num ficheiro no disco rígido. Os atacantes procuram frequentemente controlar aplicações legítimas de utilizador e ferramentas de administração do sistema, como o PowerShell e o *Windows* Management Instrumentation (WMI), para executar e propagar o *malware fileless* [36].

Posteriormente, das *frames* 153-157 o comando "CloseServiceHandle" termina o serviço. Na *frame* 159 observa-se que o cliente pede para aceder outra vez à pasta "\empty" no servidor, sendo novamente bem-sucedido. De seguida, realizou um pedido de criação do ficheiro "tGywfr0y.exe", também concretizado (*frames* 162-164). Após a criação, encontra-se uma sequência de comandos (criação e manipulação do ficheiro, seguida de encerramento abrupto) que sugere que poderia estar a ocorrer uma tentativa de execução ou manipulação remota do ficheiro "tGywfr0y.exe", nomeadamente os comandos "SetInfo Request" e "Close Request file" (*frames* 165-169).

Na *frame* 173, por sua vez, pode-se verificar a presença de uma mensagem, RST (*reset*) enviada do servidor para o cliente, indicando uma reinicialização da conexão, sendo que mesmo encerrou de forma abrupta. Tal acontece quando o sistema decide que a comunicação deve ser terminada de forma imediata, o que pode ser devido a eventos locais, como falhas de rede, ataques ou reconfigurações, bem como a tendências globais no uso do TCP [37].

Posteriormente, nas *frames* 174-178 verifica-se uma troca de informação contínua entre o cliente e o servidor. Nas *frames* 179 e 180, são observados protocolos ICMPv6, utilizados para funções de configuração e descoberta de dispositivos em redes IPv6. O comando "Router Solicitation" permite que um *host* descubra os endereços dos roteadores operacionais na sub-rede [38]. Já o "Router Advertisement" trata-se de mecanismo utilizado em redes de computadores para facilitar a configuração automática de endereços IP e outros parâmetros de rede nos dispositivos conectados [39]. A mensagem "IPv6 no next header" (*frame* 181) indica que o IPv6 não possui uma carga útil de dados, ou seja, não transporta dados adicionais. Assim, trata-se de um pacote

vazio, que provavelmente está a ser utilizado para verificar ou manter a presença do dispositivo IPv6 na rede, sem transportar dados úteis.

Na figura 56 é possível analisar a sequência de *frames* anteriormente explicada.

167	1.479092	192.168.155.226	192.168.155.232	TCP	66	45449 → 445 [ACK] Seq=24783 Ack=7169 Win=64128 Len=0 TSval=3680265056 TSecr
168	1.481266	192.168.155.226	192.168.155.232	SMB2	158	Close Request File: tGywfrOy.exe
169	1.481450	192.168.155.232	192.168.155.226	SMB2	194	Close Response
170	1.481713	192.168.155.226	192.168.155.232	TCP	66	45449 → 445 [ACK] Seq=24875 Ack=7297 Win=64128 Len=0 TSval=3680265059 TSecr
171	1.484430	192.168.155.226	192.168.155.232	TCP	66	45449 → 445 [FIN, ACK] Seq=24875 Ack=7297 Win=64128 Len=0 TSval=3680265061
172	1.484471	192.168.155.232	192.168.155.226	TCP	66	445 → 45449 [ACK] Seq=7297 Ack=24876 Win=531200 Len=0 TSval=538669 TSecr=36
173	1.484756	192.168.155.232	192.168.155.226	TCP	54	445 → 45449 [RST, ACK] Seq=7297 Ack=24876 Win=0 Len=0
174	5.573203	192.168.155.226	192.168.155.232	TCP	60	4444 → 49752 [PSH, ACK] Seq=288 Ack=167 Win=64256 Len=1
175	5.573298	192.168.155.232	192.168.155.226	TCP	55	49752 → 4444 [PSH, ACK] Seq=167 Ack=289 Win=65280 Len=1
176	5.573536	192.168.155.226	192.168.155.232	TCP	60	4444 → 49752 [ACK] Seq=289 Ack=168 Win=64256 Len=0
177	5.573850	192.168.155.232	192.168.155.226	TCP	74	49752 → 4444 [PSH, ACK] Seq=168 Ack=289 Win=65280 Len=20
178	5.574099	192.168.155.226	192.168.155.232	TCP	60	4444 → 49752 [ACK] Seq=289 Ack=188 Win=64256 Len=0
179	11.251598	fe80::ffff:ffff:fffe	ff02::2	ICMPv6	103	Router Solicitation
180	11.289360	fe80::8000:f227:d7a…	fe80::ffff:ffff:fffe	ICMPv6	151	Router Advertisement
181	11.289551	2001:0:2851:782c:3c…	ff02::1	IPv6	82	IPv6 no next header
182	14.258455	192.168.155.226	192.168.155.232	TCP	60	4444 → 49752 [PSH, ACK] Seq=289 Ack=188 Win=64256 Len=5
183	14.258559	192.168.155.232	192.168.155.226	TCP	59	49752 → 4444 [PSH, ACK] Seq=188 Ack=294 Win=65280 Len=5
184	14.258865	192.168.155.226	192.168.155.232	TCP	60	4444 → 49752 [ACK] Seq=294 Ack=193 Win=64256 Len=0
185	14.259676	192.168.155.232	192.168.155.226	TCP	56	49752 → 4444 [PSH, ACK] Seq=193 Ack=294 Win=65280 Len=2

Figura 56 - Análise das *frames* 167-185.

Posteriormente, os pacotes PSH, ACK confirmam que a comunicação na rede e transferência de dados ainda se encontra ativa (*frames* 182-201).

Nas *frames* 202 e 203, verifica-se uma comunicação ARP. O servidor faz uma solicitação, de forma a perguntar quem tem o IP 192.168.155.101. De seguida, o dispositivo com endereço 192.168.155.101 responde, informando que seu endereço MAC é 00:0c:29:66:a3. Esta ação permite deduzir que o servidor tenta estabelecer uma conexão com a máquina de IP 192.168.155.101. Como anteriormente mencionado, caso o dispositivo que fizesse o pedido ARP com o IP da vítima (192.168.155.232) mas enviando o endereço MAC do atacante, poderia tratar-se de um início de ataque *man-in-the-middle*, através de ARP *Spoofing*.

Por fim, da *frame* 204 até à 371, ocorre transferência de dados em bloco de arquivos, entre o cliente e o servidor.

Em suma, o ataque presente nesta captura passa pela criação, execução e manipulação de ficheiros, entre máquinas, sendo estes posteriormente eliminados, seguindo o ataque “malware fileless”. Assim, nesta captura o atacante é o cliente de IP 192.168.155.226 e a vítima é o servidor de IP 192.168.155.232.

3. Conclusão

Terminada a análise das 6 capturas, conclui-se que efetivamente a análise de tráfego é fundamental para o entendimento do comportamento das conexões nas redes, bem como a identificação de potenciais perigos associados a ataques cibernéticos. A ferramenta *Wireshark* mostrou-se ser uma ferramenta poderosa e essencial para a realização de uma análise de tráfego de redes detalhada e completa.

A realização deste trabalho permitiu ainda a aquisição e aprofundamento de conhecimentos relativos à Unidade Curricular (UC) de SDSIMBI, especialmente conceitos que compreendem a dinâmica das comunicações entre os dispositivos na rede, a identificação de diversos protocolos e de informações relevantes inerentes às solicitações que o utilizador faz ao longo de uma sessão. Para além disso, foi possível ainda explorar situações de ataques cibernéticos, o que permitiu reforçar a importância destas análises de tráfego de rede e identificar quais protocolos são mais suscetíveis ou possuem mais falhas de segurança, bem como quais métodos são os mais comumente utilizados pelos *hackers*.

Referências

[1] R. Love, Linux Kernel Development, PEARSON EDUCATION (US), 2010.

[2] "Red Hat Enterprise Linux," 3 janeiro 2023. [Online]. Available: https://www.redhat.com/pt-br/topics/linux/what-is-linux#o-que-%C3%A9-linux-e-para-que-ele-serve.

[3] D. W. Andrew S. Tanenbaum, Computer Networks, 5th edition, United States of America: Pearson Education, 2011.

[4] DragoHercog, "Communication Protocols: Principles, Methods and Specifications," Ljubljana, Slovenia, Springer, 2020.

[5] W. Stallings, Data and Computer Communications, 10th edition, Edinburgh Gate, Harlow, Essex CM20 2JE, England.: Pearson, 2014.

[6] "Protocolo ICMP," [Online]. Available: https://www.fortinet.com/br/resources/cyberglossary/internet-control-message-protocol-icmp . [Acedido em Novembro 2024].

[7] "Técnico Lisboa," 2024. [Online]. Available: https://si.tecnico.ulisboa.pt/servicos/recursos-e-mail/e-mail/o-que-e-o-protocolo-pop/.

[8] "Knowledge Base," [Online]. Available: https://kb.ptisp.com/o-que-e-pop3/.

[9] N. A. Mohamed, A. Jantan e O. I. Abiodun, "Protect Governments, and organizations Infrastructure against Cyber Terrorism (Mitigation and Stop of Server Message Block (SMB) Remote Code Execution Attack)," *International Journal of Engineering Research and Technology*, vol. 11, nº 2, pp. 261-272, 2018.

[10] "Microsoft - Windows Server," 4 junho 2023. [Online]. Available: https://learn.microsoft.com/pt-br/windows-server/storage/file-server/file-server-smb-overview. [Acedido em 5 novembro 2024].

[11] "Wireshark," 11 agosto 2020. [Online]. Available: https://wiki.wireshark.org/DCE/RPC. [Acedido em 2024 novembro 05].

[12] L. E. Heindel e V. A. Kasten, "Highly Reliable Synchronous and Asynchronous Remote Procedure Calls," em *Conference Proceedings of the 1996 IEEE Fifteenth Annual International Phoenix Conference on Computers and Communications*, Scottsdale, AZ, USA, 1996.

[13] "Microsoft - [MS-SCMR]: Service Control Manager Remote Protocol," 27 abril 2022. [Online]. Available: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-scmr/705b624a-13de-43cc-b8a2-99573da3635f. [Acedido em 05 novembro 2024].

[14] "TCP ou UDP: qual é a diferença e qual é o melhor protocolo?," [Online]. Available: https://www.avast.com/pt-br/c-tcp-vs-udp-difference. [Acedido em Novembro 2024].

[15] Broadcom, "VmWare," 2005- 2024 . [Online]. Available: https://www.vmware.com/topics/network-traffic-analysis.

[16] R. Tuli, "ANALYZING NETWORK PERFORMANCE PARAMETERS USING WIRESHARK," *International Journal of Network Security & Its Applications (IJNSA),* vol. 15, nº 1, 2023.

[17] M. A. B. A. H. M. Naif Alsharabi, "Detecting Unusual Activities in Local NetworkUsing Snort and Wireshark Too," *Journal of Advances in Information Technology,* vol. 14, nº 4, 2023.

[18] B. B. Sudhanshu Sekhar Tripathy, "EVALUATION OF FUTURE PERSPECTIVES ON SNORT AND WIRESHARK AS TOOLS AND TECHNIQUES FOR INTRUSION DETECTION SYSTEM," *Industrial Engineering Journal,* vol. 53, nº 2, 10 2024.

[19] "Controle Net," [Online]. Available: https://www.controle.net/faq/cliente-servidor-uma-estrutura-para-a-computacao-centralizada.

[20] Y.-L. H. C.-Y. T. W.-T. C. C.-H. L. e. K. C. Fu-Hau Hsu, "TRAP: A Three-Way Handshake Server for TCP," *Applied Sciences,* vol. 6, nº 11, p. 358, 2016.

[21] "Mundo Unix," 31 outubro 2016. [Online]. Available: https://www.mundounix.net/informacoes-detalhadas-de-arquivosdiretorios/.

[22] B. P. Godfrey, "Billpg Industries," 17 dezembro 2020. [Online]. Available: https://billpg.com/pop3-uidl/. [Acedido em 1 novembro 2024].

[23] "Data Motion," 15 dezembro 2020. [Online]. Available: https://kb.datamotion.com/?ht_kb=how-to-utilize-pop3-protocol-to-access-datamotion-securemail-and-direct-accounts. [Acedido em 1 novembro 2024].

[24] "NetBIOS," [Online]. Available: https://pt.wikipedia.org/wiki/NetBIOS. [Acedido em Novembro 2024].

[25] "NetBIOS Suffix Definitions," [Online]. Available: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-brws/0c773bdd-78e2-4d8b-8b3d-b7506849847b. [Acedido em Novembro 2024].

[26] L. Chappel, Wireshark Network Analysis, The Official Wireshark Certified Network Analyst Study Guide, 2nd edition, San Jose, California, USA: Protocol Analysis Institute, Inc., 2012.

[27] J. P. e. J. Reynolds, "RFC 959 - File Transfer Protocol (FTP), Internet Engineering Task Force (IETF)," outubro 1985. [Online]. Available: https://www.rfc-editor.org/rfc/rfc959.

[28] RFC Editor, "RFC 775 - File Transfer Protocol (FTP): Control Connection Extensions," july 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc775.html..

[29] "Use the PowerShell Passthru Parameter and Get Back Objects," Microsoft, 18 november 2011. [Online]. Available: https://devblogs.microsoft.com/scripting/use-the-powershell-passthru-parameter-and-get-back-objects/.

[30] "What is User Agent Spoofing?," Peakhour, 2024. [Online]. Available: https://www.peakhour.io/learning/bots/user-agent-spoofing/.

[31] J. P. Jessey Bullock, Wireshark for Security Professionals: Using Wireshark and the Metasploit Framework, Wiley, 2017.

[32] M. Data, "The defense against ARP spoofing attack using semi-static ARP cache table," *International Conference on Sustainable Information Engineering and Technology* , pp. 206-210, 2018.

[33] B. Bhushan, G. Sahoo e A. K. Rai, "Man-in-the-middle attack in wireless and computer networking — A review," em *2017 3rd International Conference on Advances in Computing,Communication & Automation (ICACCA) (Fall)*, 2017.

[34] "Microsoft - Interprocess communications," 02 fevereiro 2024. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/ipc/interprocess-communications. [Acedido em 02 novembro 2024].

[35] "Microsoft - SMB2 TREE_DISCONNECT Response," 26 fevereiro 2024. [Online]. Available: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/aeac92de-8db3-48f8-a8b7-bfee28b9fd9e. [Acedido em 05 novembro 2024].

[36] B. Sanjay, D. Rakshith, R. Akash e V. V.Hegde, "An Approach to Detect Fileless Malware and Defend its Evasive mechanisms," em *3rd IEEE International Conference on Computational Systems and Information Technology for Sustainable Solutions*, 2018.

[37] M. Arlitt e C. Williamson, "An Analysis of TCP Reset Behaviour on the Internet," *ACM SIGCOMM Computer Communications Review*, vol. 35, nº 1, pp. 37-44, 2005.

[38] "Juniper Networks," 20 janeiro 2021. [Online]. Available: https://www.juniper.net/documentation/us/en/software/junos/icmp/topics/concept/icmp-routing-router-discovery-overview.html. [Acedido em 07 novembro 2024].

[39] "ExpressVPS," 1 janeiro 2024. [Online]. Available: https://expressvps.com.br/glossario/o-que-e-router-advertisement/. [Acedido em 5 novembro 2024].

[40] A. Mallik, "Man-in-the-middle-attack: understanding in simple words," *Jurnal Pendidikan Teknologi Informasi* , vol. 2, nº 2, pp. 109-134, outubro 2018.